



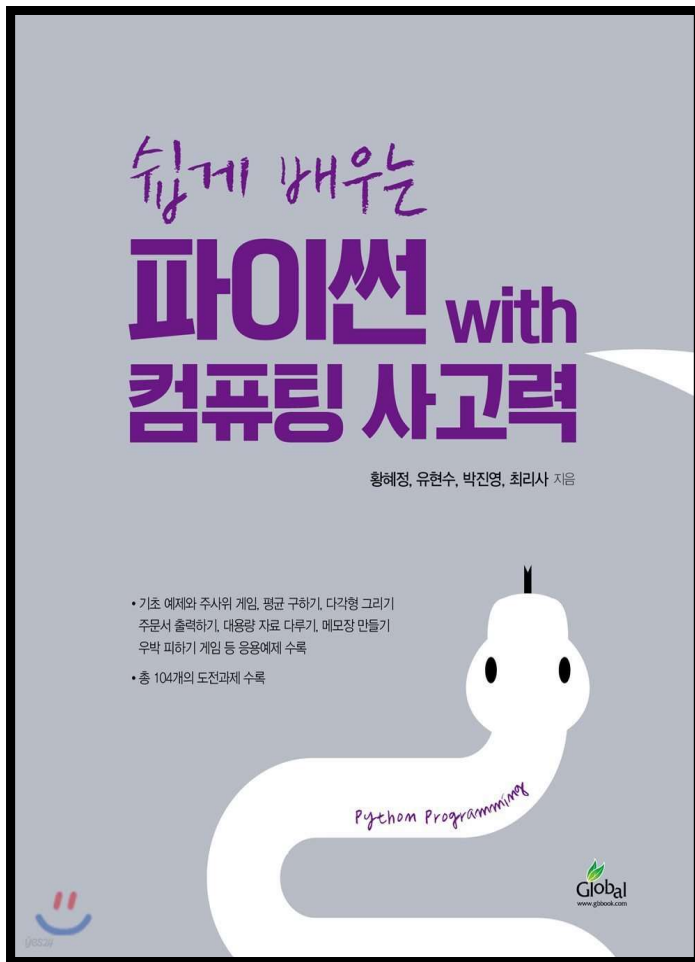
광운대학교
KwangWoon University

PYTHON

코딩강사과정

겸임교수 박세영

도서 추천



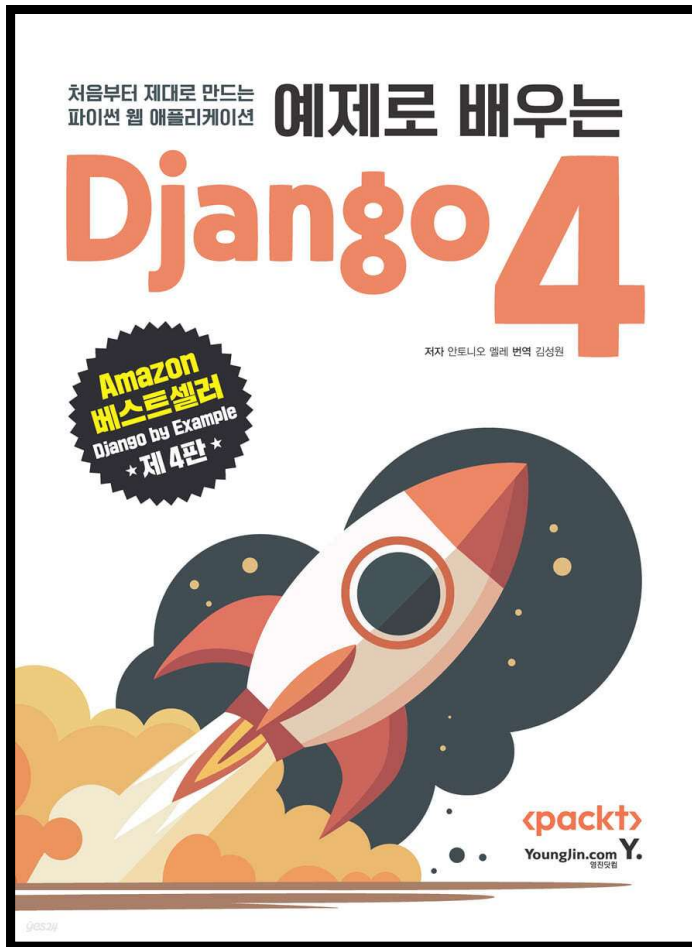
- **도서명:** 쉽게 배우는 파이썬
with 컴퓨팅 사고력
- **ISBN:** 9788955027389
- **저자:** 황혜정, 유현수, 박진영
- **출판사:** 글로벌
- **페이지 / 정가:** 409p / 18,000원

도서 추천



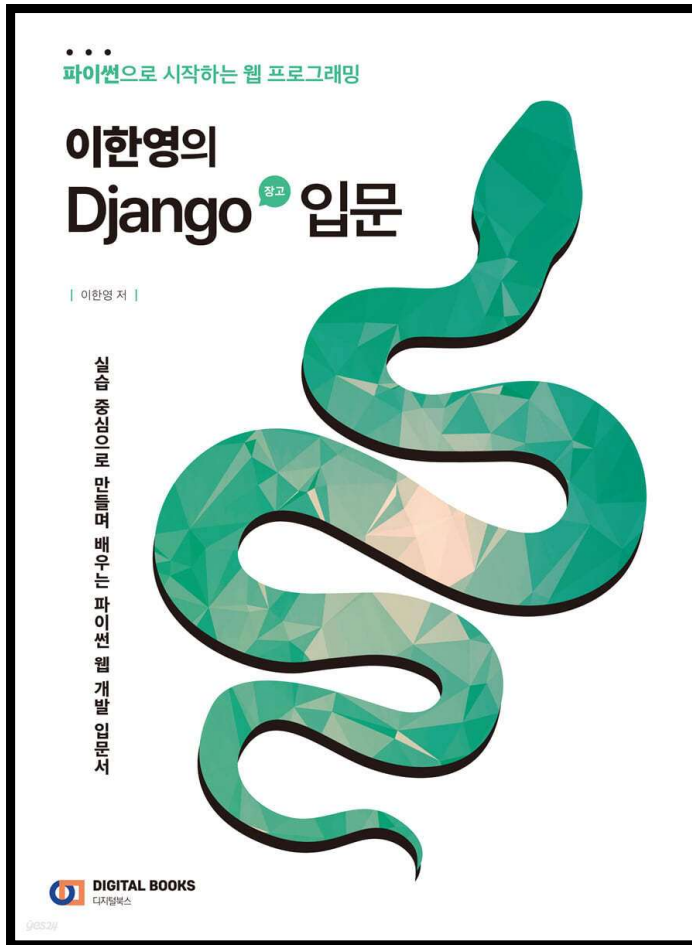
- **도서명:** 파이썬 for Beginner
- **ISBN:** 9791156645740
- **저자:** 우재남
- **출판사:** 한빛아카데미
- **페이지 / 정가:** 524p / 25,000원

도서 추천



- **도서명:** 예제로 배우는 Django 4
- **ISBN:** 9788931474435
- **저자:** 안토니오 멜레
- **출판사:** 영진닷컴
- **페이지 / 정가:** 856p / 34,200원

도서 추천



- **도서명:** 이한영의 Django 입문
- **ISBN:** 9788960884229
- **저자:** 이한영
- **출판사:** 디지털북스
- **페이지 / 정가:** 384p / 22,500원

목차

1. Django 웹 개념 및 개발환경
2. 블로그 어플리케이션 만들기
3. 고급 기능으로 블로그 향상시키기

01

Django 웹 개념 및 개발환경

01. Django 웹 개념 및 개발환경

■ Django란?

- 파이썬으로 작성된 웹 프레임워크
- 프레임워크란 반복적으로 사용되는 특정 기술을 모아놓은 도구 상자와 같은 기능
- 웹 사이트를 만들기 위해 일반적으로 필요로 하는 기능들을 제공



01. Django 웹 개념 및 개발환경

■ Django의 핵심기능

- 데이터베이스 관리
- 강력한 관리자 기능
- 보안 기능
- 다양한 내장 기능
- 파이썬 확장 기능



01. Django 웹 개념 및 개발환경

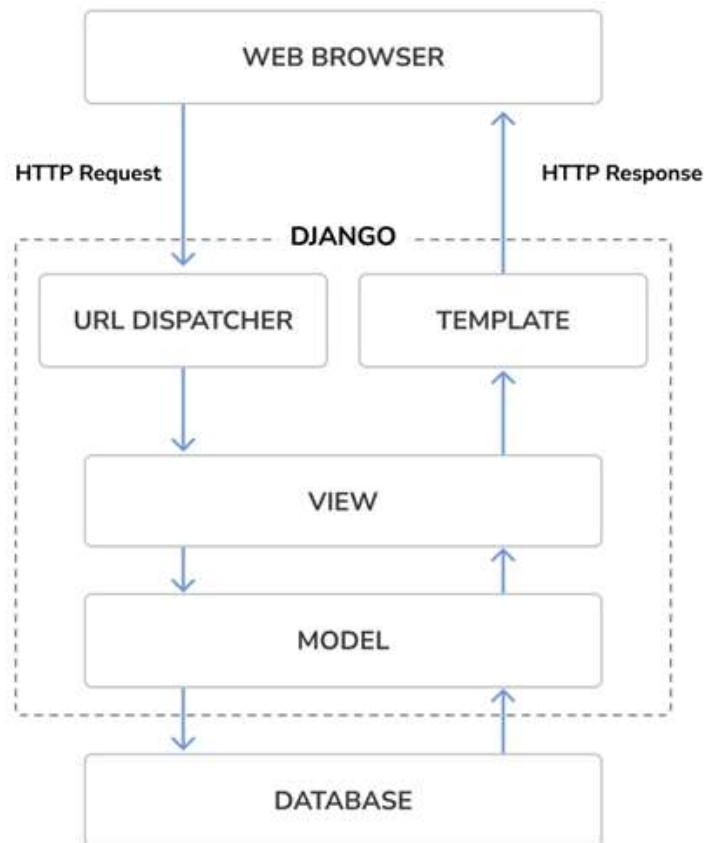
■ Django의 주요 프레임워크

- Django는 MVC(Model-View-Controller)패턴과 유사한 패턴인 MTV(Model-Template-View) 패턴이다.
- 모델 : 논리적 데이터 구조를 정의하는 데이터베이스와 뷰 사이의 데이터 핸들러를 의미한다.
- 템플릿 : 프레젠테이션 계층으로 브라우저가 렌더링하는 모든 것을 가진 일반 텍스트 템플릿 시스템을 사용한다.
- 뷰 : 모델을 통해 데이터베이스와 통신하고 뷰를 위해 데이터를 템플릿으로 전송한다.
- Django 프로젝트를 개발할때는 항상 모델, 뷰, 템플릿 및 URL의 구성으로 작업이 이루어진다.

01. Django 웹 개념 및 개발환경

■ 웹사이트가 화면에 실행되는 과정

- URL, 뷰, 모델 그리고 템플릿과 같은 Django의 주요 컴포넌트들을 사용해서 요청/응답 사이클을 관리하는 방법은 다음과 같다.



1. 웹 브라우저가 URL로 페이지를 요청하고 웹 서버가 HTTP 요청을 장고에 전달한다.
2. Django는 설정된 URL 패턴을 살펴보고 처음에 매치되는 URL 패턴을 발견한다.
3. Django는 일치하는 URL 패턴에 해당하는 뷰를 실행한다.
4. 뷰는 가능한 데이터 모델을 사용해서 데이터베이스로부터 정보를 검색한다.
5. 데이터 모델은 데이터 정의 및 동작을 제공한다. 데이터베이스에 질의하는데 사용한다.
6. 뷰는 템플릿을 렌더링하여 데이터를 표시하고 HTTP 응답과 함께 반환한다.

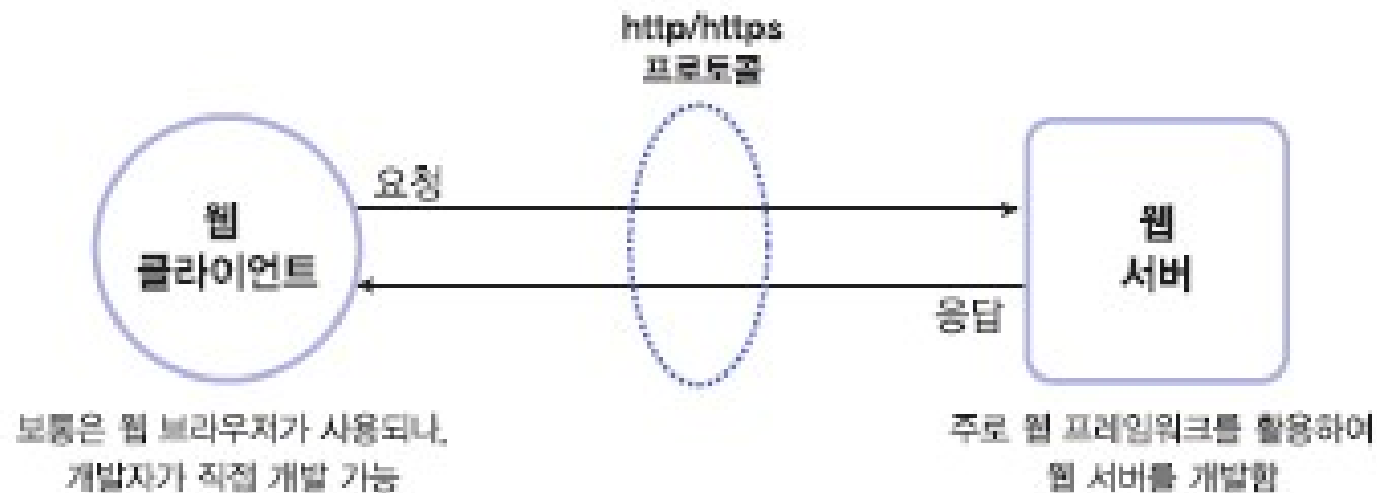
01. Django 웹 개념 및 개발환경

■ 웹 프로그래밍의 이해

HTTP(S) 프로토콜로 통신하는, 클라이언트와 서버를 개발

웹 클라이언트와 웹 서버를 같이 개발 및 웹 클라이언트 또는 웹 서버 하나만 개발

웹 서버를 개발하는 경우가 많아서 파이썬 웹 프로그래밍이라고 하면 우선적으로 생각하고 Django와 같은 웹 프레임워크를 사용하여 웹 서버를 개발



01. Django 웹 개념 및 개발환경

■ 다양한 웹 클라이언트

1.2.1 웹 브라우저를 사용하여 요청



브라우저는 주소창에 입력된 문장을 해석하여 웹 서버에게 HTTP 요청을 보내는 웹 클라이언트의 역할을 수행한다.

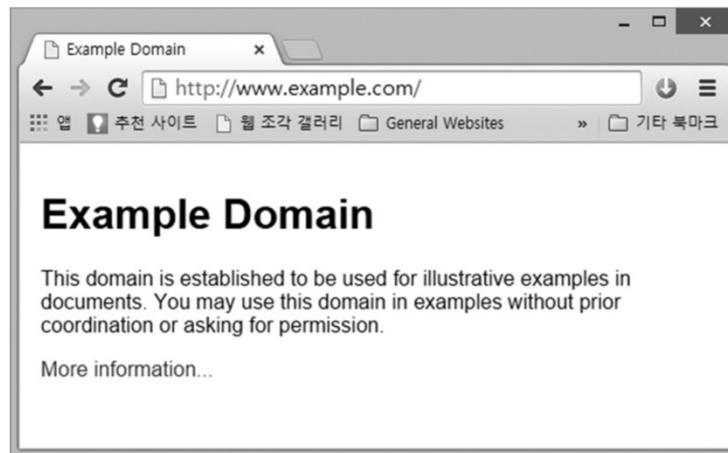


그림 1-3 웹 브라우저 요청에 대한 웹 서버의 응답

01. Django 웹 개념 및 개발환경

■ 다양한 웹 클라이언트

1.2.2 리눅스 curl 명령을 사용하여 요청

리눅스 curl 명령은 HTTP/HTTPS/FTP 등 여러 가지의 프로토콜을 사용하여 데이터를 송수신할 수 있는 명령

```
$ curl http://www.example.com|
```

```
[shkim@localhost chi]$ curl http://www.example.com
<doctype html>
<html>
<head>
  <title>Example Domain</title>
  ... (중략)

</head>

<body>
<div>
  <h1>Example Domain</h1>
  <p>This domain is established to be used for illustrative examples in
documents. You may use this domain in examples without prior coordination or
asking for permission.</p>
  <p><a href="http://www.iana.org/domains/example">More information...</a></p>
</div>
</body>
</html>
[shkim@localhost chi]$
```

그림 1-4 curl 명령 요청에 대한 웹 서버의 응답

[그림 1-3]과 비교해서 HTML 태그들과 <head> 부분이 더 보이지만 이는 동일한 응답

01. Django 웹 개념 및 개발환경

■ 다양한 웹 클라이언트

1.2.3 Telnet을 사용하여 요청

리눅스의 telnet 프로그램을 사용하여 HTTP 요청을 보낼 수도 있음

```
[shkin@localhost ch1]$ telnet www.example.com 80
Trying 93.184.216.119...
Connected to www.example.com.
Escape character is '^]'.
GET / HTTP/1.1  입력
Host: www.example.com  입력
 입력
```

telnet 명령은 터미널 창에서 입력하는 내용을 그대로 웹 서버에 전송

telnet 프로그램이 웹 클라이언트의 역할을 수행

01. Django 웹 개념 및 개발환경

■ 다양한 웹 클라이언트

1.2.3 Telnet을 사용하여 요청

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: max-age=604800
Content-Type: text/html
Date: Sun, 09 Nov 2014 11:46:41 GMT
Etag: "359670651"
Expires: Sun, 16 Nov 2014 11:46:41 GMT
Last-Modified: Fri, 09 Aug 2013 23:54:35 GMT
Server: ECS (rhv/S1BF)
X-Cache: HIT
x-ec-custom-error: 1
Content-Length: 1270
```

```
<!doctype html>
<html>
<head>
  <title>Example Domain</title>
  ... (중략)
```

```
</head>
```

```
<body>
```

```
<div>
```

```
  <h1>Example Domain</h1>
  <p>This domain is established to be used for illustrative examples in
  documents. You may use this domain in examples without prior coordination or
  asking for permission.</p>
  <p><a href="http://www.iana.org/domains/example">More information...</a></p>
</div>
</body>
</html>
~]
telnet> quit Enter 입력
Connection closed.
[shkim@localhost ch1]$
```

응답 메시지를 수신한 후, telnet 프로그램을 종료하기 위해 마지막 두 라인을 입력한 것에 유의

동일한 응답을 확인할 수 있음

그림 1-5 telnet 명령 요청에 대한 웹 서버의 응답

01. Django 웹 개념 및 개발환경

■ 다양한 웹 클라이언트

1.2.4 직접 만든 클라이언트로 요청

```
C:\WRedBook\wch1>notepad example.py
```

```
import urllib.request
print(urllib.request.urlopen('http://www.example.com').read().decode('utf-8'))
```

```
C:\WRedBook\wch1>python example.py
<!doctype html>
<html>
<head>
  <title>Example Domain</title>
  ... (중략)
</head>
<body>
<div>
  <h1>Example Domain</h1>
  <p>This domain is established to be used for illustrative examples in
documents. You may use this domain in examples without prior coordination or
asking for permission.</p>
  <p><a href="http://www.iana.org/domains/example">More information...</a></p>
</div>
</body>
</html>
C:\WRedBook\wch1>
```

그림 1-6 직접 만든 클라이언트 요청에 대한 웹 서버의 응답

앞에서 보았던 응답 결과와 동일한 것을 확인할 수 있음.

즉, 웹 클라이언트의 형태는 달라도 동일한 요청에 대해서 동일한 응답을 받는것을 확인

01. Django 웹 개념 및 개발환경

■ HTTP 프로토콜

1.3.1 HTTP 메시지의 구조

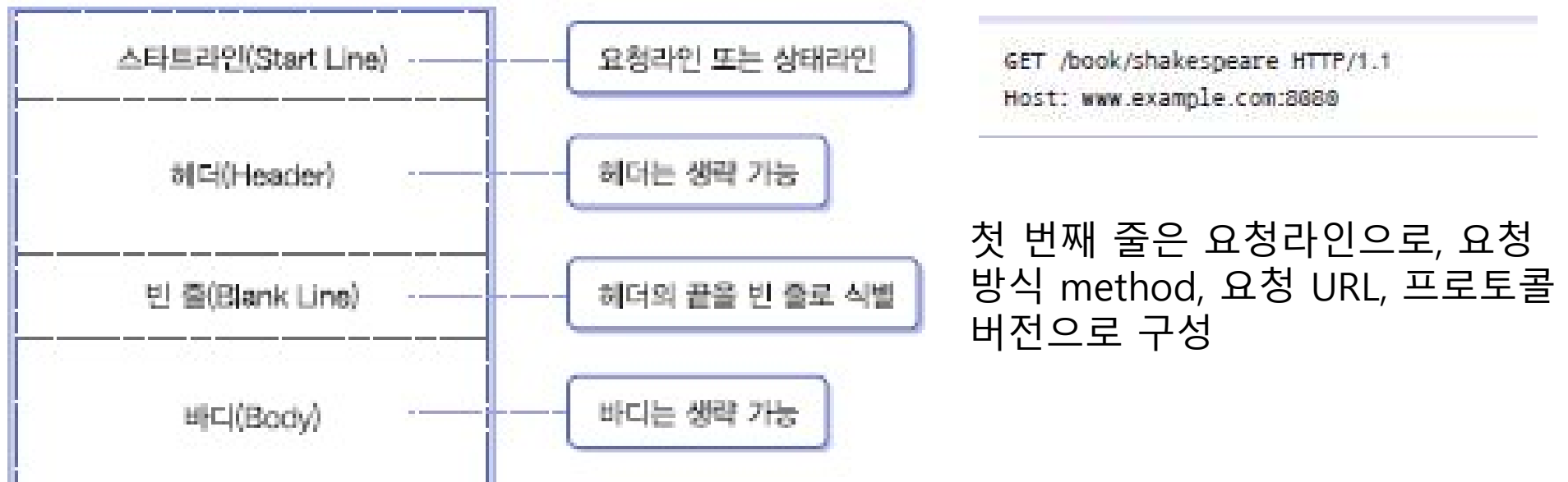


그림 1-7 HTTP 메시지 구조

[그림 1-7]에서 스타트라인은 요청 메시지일 때 요청라인 request line이라고 하고, 응답 메시지일 때 상태라인 status line이라고 함

01. Django 웹 개념 및 개발환경

■ HTTP 프로토콜

1.3.2 HTTP 처리 방식

메소드명	의미	CRUD와 매핑되는 역할
GET	리소스 취득	Read(조회)
POST	리소스 생성 리소스 데이터 추가	Create(생성)
PUT	리소스 변경	Update(변경)
DELETE	리소스 삭제	Delete(삭제)
HEAD	리소스의 헤더(메타데이터) 취득	
OPTIONS	리소스가 서포트하는 메소드 취득	
TRACE	루프백 시험에 사용	
CONNECT	프록시 동작의 터널 접속으로 변경	

표 1-1 HTTP 메소드 종류

GET 방식은 지정한 URL의 정보를 가져오는 메소드로, 가장 많이 사용

POST의 대표적인 기능은 리소스를 생성하는 것으로, 블로그에 글을 등록하는 경우가 이에 해당. PUT은 리소스를 변경하는 데 사용

01. Django 웹 개념 및 개발환경

■ HTTP 프로토콜

1.3.3 GET과 POST 메소드

앞에서 8가지의 HTTP 메소드를 소개하였지만, 현실적으로 가장 많이 사용하는 메소드는 GET과 POST 2가지

```
POST http://docs.djangoproject.com/search/ HTTP/1.1
Content-Type: application/x-www-form-urlencoded

q=forms&release=1
```

```
GET http://docs.djangoproject.com/search/?q=forms&release=1 HTTP/1.1
```

파이썬의 장고 프레임워크에서도 폼의 데이터는 POST 방식만을 사용하고 있음

01. Django 웹 개념 및 개발환경

■ HTTP 프로토콜

1.3.3 GET과 POST 메소드

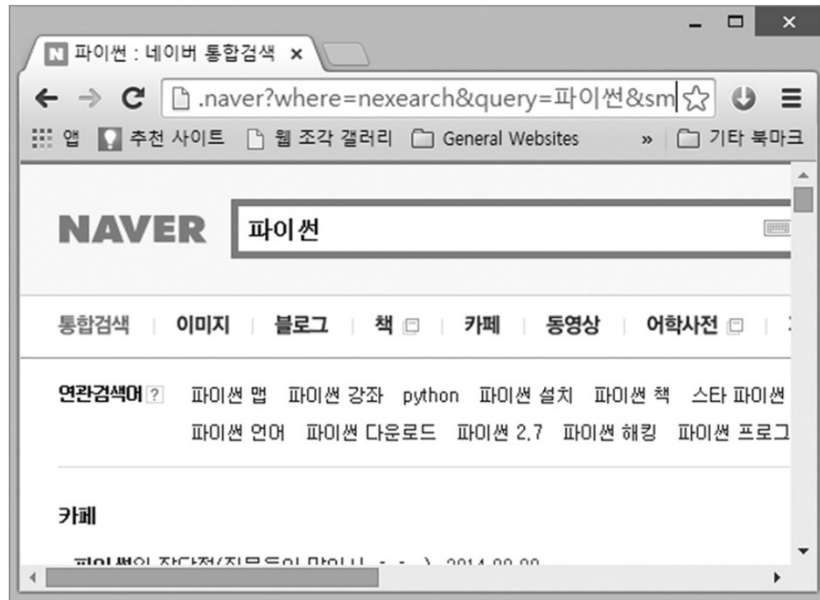


그림 1-8 네이버의 검색 창 – GET 방식 전달

01. Django 웹 개념 및 개발환경

■ HTTP 프로토콜

1.3.4 상태 코드

메시지명	의미	CRUD와 매핑되는 역할
1xx	Informational (정보 제공)	임시적인 응답으로, 현재 클라이언트의 요청까지 처리되었으니 계속 진행하라는 의미입니다. HTTP 1.1 버전부터 추가되었습니다.
2xx	Success(성공)	클라이언트의 요청이 서버에서 성공적으로 처리되었다는 의미입니다.
3xx	Redirection (리다이렉션)	완전한 처리를 위해서 추가적인 동작을 필요로 하는 경우입니다. 주로 서버의 주소 또는 요청한 URI의 웹 문서가 이동되었으니, 그 주소로 다시 시도해보라는 의미입니다.
4xx	Client Error (클라이언트 에러)	없는 페이지를 요청하는 것처럼 클라이언트의 요청 메시지 내용이 잘못된 경우입니다.
5xx	Server Error (서버 에러)	서버 측 사정에 의해서 메시지 처리에 문제가 발생한 경우입니다. 서버의 부하, DB 처리 과정 오류, 서버에서 익셉션이 발생하는 경우가 이에 해당합니다.

표 1-2 상태 코드 분류

서버에서의 처리 결과는 응답 메시지의 상태라인에 있는 상태 코드 **Status code**를 보고 파악할 수 있음

01. Django 웹 개념 및 개발환경

■ HTTP 프로토콜

1.3.4 상태 코드

상태 코드	상태 텍스트	응답 문구	서버 측면에서의 의미
2xx	Success	성공	클라이언트가 요청한 동작을 수신하여 이해했고, 승낙했으며 성공적으로 처리했다.
200	OK	성공	서버가 요청을 성공적으로 처리했다.
201	Created	생성됨	요청이 처리되어서 새로운 리소스가 생성되었다. 응답 헤더 Location에 새로운 리소스의 절대 URL을 기록합니다.
202	Accepted	허용됨	요청은 접수했지만 처리가 완료되지 않았다. 클라이언트는 응답 헤더의 Location, Retry-After를 참고하여 다시 요청을 보냅니다.
3xx	Redirection	리다이렉션	클라이언트는 요청을 마치기 위해 추가적인 동작을 취해야 한다.
301	Moved Permanently	영구 이동	지정된 리소스가 새로운 URI로 이동했다. 이동할 곳의 새로운 URI는 응답 헤더 Location에 기록합니다.

표 1-3 자주 사용되는 상태 코드

01. Django 웹 개념 및 개발환경

■ HTTP 프로토콜

1.3.4 상태 코드

상태 코드	상태 텍스트	응답 문구	서버 측면에서의 의미
303	See Other	다른 위치 보기	다른 위치로 요청하라. 요청에 대한 처리 결과를 응답 헤더 Location에 표시된 URI에서 GET으로 취득할 수 있습니다. 브라우저의 폼 요청을 POST로 처리하고 그 결과 화면으로 리다이렉트시킬 때 자주 사용하는 응답 코드입니다.
307	Temporary Redirect	임시 리다이렉션	임시로 리다이렉션 요청이 필요하다. 요청한 URI가 없으므로, 클라이언트는 메소드를 그대로 유지한 채 응답 헤더 Location에 표시된 다른 URI로 요청을 재송신할 필요가 있습니다. 클라이언트는 향후 요청 시 원래 위치를 계속 사용해야 합니다. 302의 의미를 정확하게 재정의해서 HTTP/1.1의 307 응답으로 추가되었습니다.
4xx	Client Error	클라이언트 에러	클라이언트의 요청에 오류가 있다.
400	Bad Request	잘못된 요청	요청의 구문이 잘못되었다. 클라이언트가 모르는 4xx 계열의 응답 코드가 반환된 경우에도 클라이언트는 400과 동일하게 처리하도록 규정하고 있습니다.
401	Unauthorized	권한 없음	지정한 리소스에 대한 액세스 권한이 없다. 응답 헤더 WWW-Authenticate에 필요한 인증 방식을 지정합니다.
403	Forbidden	금지됨	지정한 리소스에 대한 액세스가 금지되었다. 401 인증 처리 이외의 사유로 리소스에 대한 액세스가 금지되었음을 의미합니다. 리소스의 존재 자체를 은폐하고 싶은 경우는 404 응답 코드를 사용할 수 있습니다.
404	Not Found	찾을 수 없음	지정한 리소스를 찾을 수 없다.
5xx	Server Error	서버 에러	클라이언트의 요청은 유효한데, 서버가 처리에 실패했다.
500	Internal Server Error	내부 서버 오류	서버쪽에서 에러가 발생했다. 클라이언트가 모르는 5xx 계열의 응답 코드가 반환된 경우에도 클라이언트는 500과 동일하게 처리하도록 규정하고 있습니다.
502	Bad Gateway	불량 게이트웨이	게이트웨이 또는 프록시 역할을 하는 서버가 그 뒷단의 서버로부터 잘못된 응답을 받았다.
503	Service Unavailable	서비스 제공불가	현재 서버에서 서비스를 제공할 수 없다. 보통은 서버의 과부하나 서비스 점검 등 일시적인 상태입니다.

01. Django 웹 개념 및 개발환경

■ URL 설계



그림 1-9 URL 구성 항목

URL 스킴 : URL에 사용된 프로토콜을 의미.

- 호스트명 : 웹 서버의 호스트명으로, 도메인명 또는 IP 주소로 표현.
- 포트번호 : 웹 서버 내의 서비스 포트번호. 생략 시에는 디폴트 포트번호로, http는 80을, https는 443을 사용
- 경로 : 파일이나 애플리케이션 경로를 의미
- 쿼리스트링 : 질의 문자열로, 앰퍼샌드(&)로 구분된 이름=값 쌍 형식으로 표현
- 프래그먼트 : 문서 내의 앵커 등 조각을 지정

01. Django 웹 개념 및 개발환경

■ URL 설계

1.4.1 URL을 바라보는 측면

URL은 웹 클라이언트에서 호출한다는 시점에서 보면, 웹 서버에 존재하는 애플리케이션에 대한 **API**Application Programming Interface라고 할 수 있음

```
http://blog.example.com/search?q=test&debug=true
```

URL을 바라보는 또 한 가지 측면은 REST 방식으로 URL을 설계하는 것.
REST 방식이란 웹 서버에 존재하는 요소들을 모두 리소스라고 정의하고,
URL을 통해 웹 서버의 특정 리소스를 표현한다는 개념.

01. Django 웹 개념 및 개발환경

■ URL 설계

1.4.2 간편 URL 표 1-4 기존 URL과 간편 URL 간의 비교를 위한 대응 예시

기존 URL	간편 URL
<code>http://example.com/index.php?page=foo</code>	<code>http://example.com/foo</code>
<code>http://example.com/index.php?page=consulting/marketing</code>	<code>http://example.com/consulting/marketing</code>
<code>http://example.com/products?category=2&pid=25</code>	<code>http://example.com/products/2/25</code>
<code>http://example.com/cgi-bin/feed.cgi?feed=news&fm=rss</code>	<code>http://example.com/news.rss</code>
<code>http://example.com/services/index.jsp?category=legal&id=patents</code>	<code>http://example.com/services/legal/patents</code>
<code>http://example.com/index.asp?mod=profiles&id=199</code>	<code>http://example.com/user/john-doe</code>
<code>http://example.com/app/dashboard/dspchr_c80.dll?page=38661&mod1=bnr_ant&UID=4511681&SESSIONID=4fd8b561ac867195fba2cc5679&...</code>	<code>http://example.com/app/dashboard/reports#monthly</code>

간편 URL은 쿼리스트링 없이 경로만 가진 간단한 구조의 URL을 말함.
검색 엔진의 처리를 최적화하기 위해 생겨난 간편한 URL은 URL을 입력하거나 기억하기 쉽다는 부수적인 장점도 있어, 검색 엔진 친화적 URL_{search engine friendly url} 또는 사용자 친화적 URL_{user friendly url}이라고 부르기도 함

01. Django 웹 개념 및 개발환경

■ URL 설계

1.4.3 파이썬의 우아한 URL

```
urlpatterns = [  
    path('articles/2003/', views.special_case_2003),  
    path('articles/<int:year>/', views.year_archive),  
    path('articles/<int:year>/<int:month>/', views.month_archive),  
    path('articles/<int:year>/<int:month>/<slug:slug>/', views.article_detail),  
]
```

```
urlpatterns = [  
    path('articles/2003/', views.special_case_2003),  
    re_path(r'^articles/(?P<year>[0-9]{4})/$', views.year_archive),  
    re_path(r'^articles/(?P<year>[0-9]{4})/(?P<month>[0-9]{2})/$', views.month_archive),  
    re_path(r'^articles/(?P<year>[0-9]{4})/(?P<month>[0-9]{2})/(?P<slug>[ww-]+)/$', views.  
    article_detail),  
]
```

URL을 정의하기 위해 정규표현식 Regular Expression을 추가적으로 사용할 수 있음.

01. Django 웹 개념 및 개발환경

■ 웹 애플리케이션 서버

구분	역할	프로그램 명
웹 서버	웹 클라이언트의 요청을 받아서 요청을 처리하고 그 결과를 웹 클라이언트에게 응답합니다. 주로 정적 페이지인 HTML, 이미지, CSS, 자바스크립트 파일을 웹 클라이언트에 제공할 때 웹 서버를 사용합니다. 만약 동적 페이지 처리가 필요하다면 웹 애플리케이션 서버에 처리를 넘깁니다.	Apache httpd, Nginx, lighttpd, IIS 등
웹 애플리케이션 서버	웹 서버로부터 동적 페이지 요청을 받아서 요청을 처리하고, 그 결과를 웹 서버로 반환합니다. 주로 동적 페이지 생성을 위한 프로그램 실행과 데이터베이스 연동 기능을 처리합니다.	Apache Tomcat, JBoss, WebLogic, WebSphere, Jetty, Jous, mod_wsgi, uWSGI, Gunicorn 등

웹 서버 및 웹 애플리케이션 서버라는 용어는 SW 측면의 서버 프로그램을 의미

01. Django 웹 개념 및 개발환경

■ 웹 애플리케이션 서버

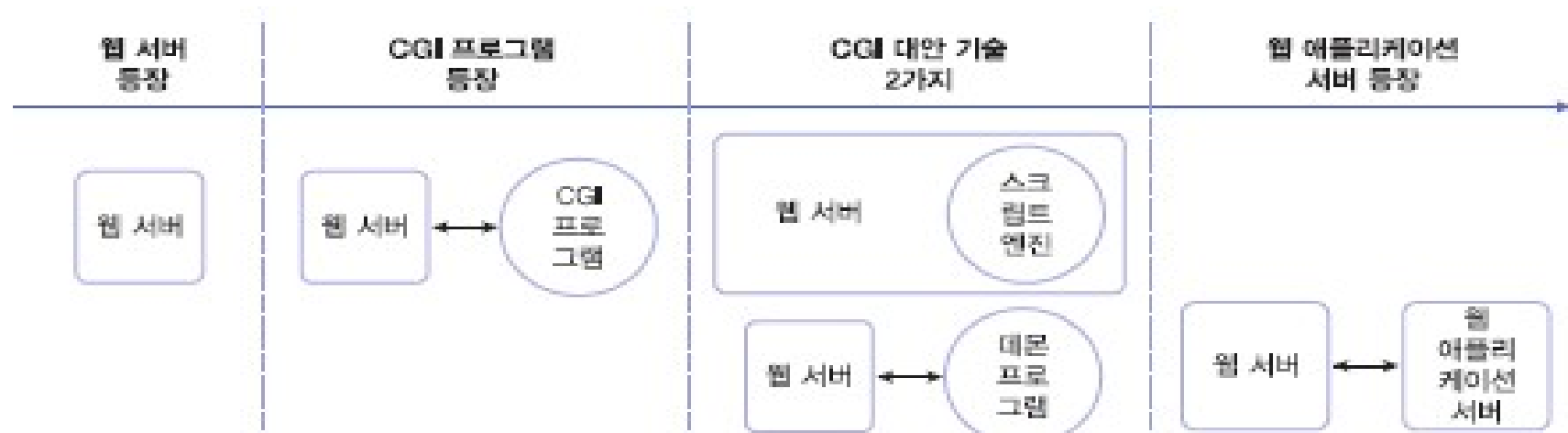


그림 1-10 기술의 발전에 따른 웹 서버 기술의 변화

01. Django 웹 개념 및 개발환경

■ 웹 애플리케이션 서버

1.5.1 정적 페이지 vs 동적 페이지

정적_{static}, 동적_{dynamic}이란 용어는 사용자가 페이지를 요청하는 시점에 페이지의 내용이 유지되는가 또는 변경되는가를 구분해주는 용어.

동적 페이지에는 프로그래밍 코드가 포함되어 있어서 페이지 요청 시점에 HTML 문장을 만들어내는 것

01. Django 웹 개념 및 개발환경

■ 웹 애플리케이션 서버

1.5.2 CGI 방식의 단점

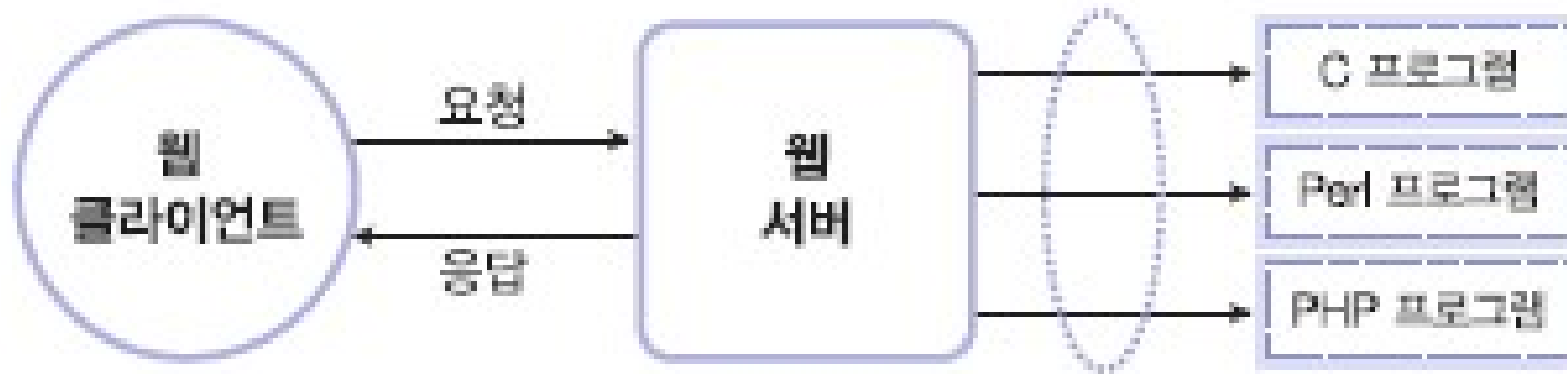


그림 1-11 전통적인 CGI 방식의 요청 처리적인 프로세스로 실행

CGI 방식의 근본적인 문제점은 각각의 클라이언트 요청에 대하여 독립적인 별도의 프로세스가 생성

요청이 많아질수록 프로세스가 많아지고, 프로세스가 많아질수록 비례적으로 프로세스가 점유하는 메모리 요구량도 커져서 시스템에 많은 부하를 주는 요인

01. Django 웹 개념 및 개발환경

■ 웹 애플리케이션 서버

1.5.3 CGI 방식의 대안 기술

CGI 방식의 대안 기술 중 하나는 별도의 애플리케이션(CGI 프로그램과 같은 역할을 하는 프로그램)을 Perl, PHP 등의 스크립트 언어로 작성하고, 스크립트를 처리하는 스크립트 엔진(인터프리터)을 웹 서버에 내장시켜서 CGI 방식의 단점이었던 별도의 프로세스를 기동시키는 오버헤드를 줄이는 방식

01. Django 웹 개념 및 개발환경

■ 웹 애플리케이션 서버

1.5.4 애플리케이션 서버 방식

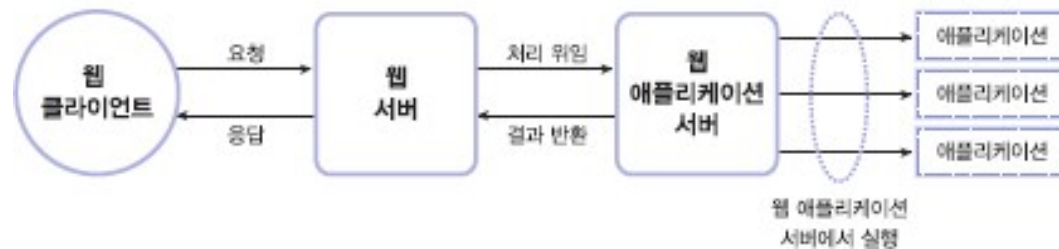


그림 1-12 애플리케이션 서버 방식의 요청 처리

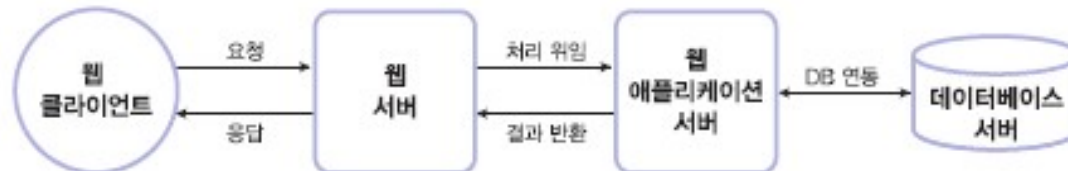


그림 1-13 애플리케이션 서버 방식에서의 서버 간 구성도

웹 서버와 웹 애플리케이션 서버가 분리됨에 따라, 서로의 역할도 구분하여 사용하는 것이 좋음. 왜냐하면 정적 페이지를 처리하는 경우에 비해서 동적 페이지를 처리하는 경우가 수 배에서 수십 배의 메모리를 소비하기 때문.

01. Django 웹 개념 및 개발환경

■ 웹 애플리케이션 서버

1.5.5 웹 서버와의 역할 구분

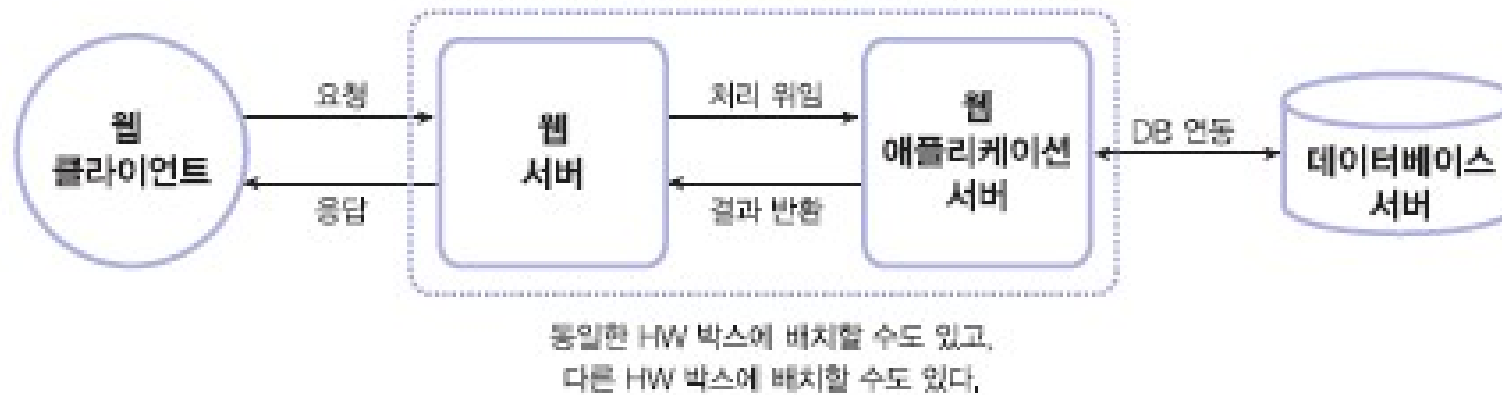


그림 1-14 웹 서버와 애플리케이션 서버의 역할과 HW 배치

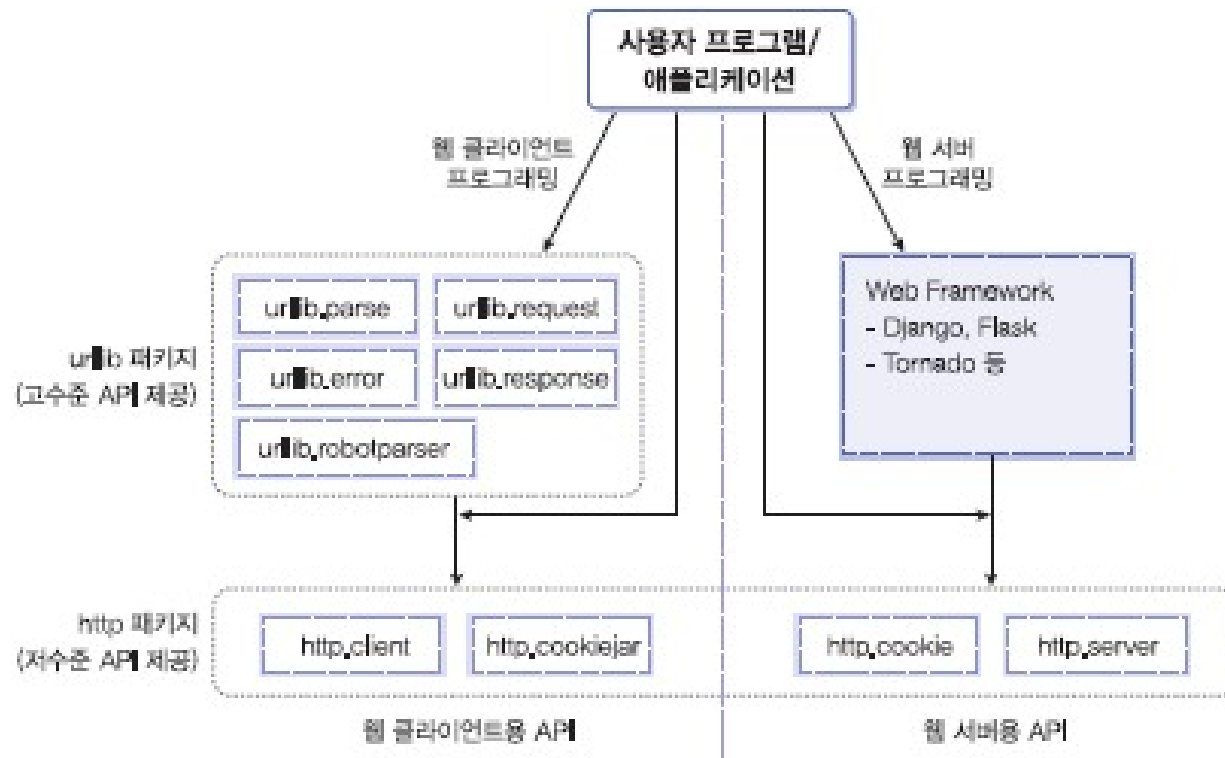
지금까지 웹 서버 및 웹 애플리케이션 서버라는 용어를 사용하였는데, 이는 SWSsoftware 측면의 서버 프로그램을 의미

웹 서버와 웹 애플리케이션 서버 프로그램이 함께 필요하며, 이 두 개의 서버를 동일한 HW 박스에서 기동시키는 것도 충분히 가능한 구성.
서비스 운용 관리 측면에서 하나의 HW 박스에 구성하는 것이 좀 더 간편한 방식이기 때문

01. Django 웹 개념 및 개발환경

■ 웹 라이브러리 구성

2.1 웹 라이브러리 구성



`urllib` 패키지에는 웹 클라이언트를 작성하는 데 사용되는 모듈들이 있으며, 가장 빈번하게 사용하는 모듈.

`http` 패키지는 크게 서버용과 클라이언트용 라이브러리로 나누어 모듈을 담고 있으며, 쿠키 관련 라이브러리도 `http` 패키지 내에서 서버용과 클라이언트용으로 모듈이 구분

01. Django 웹 개념 및 개발환경

■ 웹 라이브러리 구성

표 2-1 파이썬 3.x에서 표준 라이브러리의 모듈 구성 변경사항

파이썬 3.x 모듈명	파이썬 2.x 모듈명	파이썬 3.x에서의 변화
urllib.parse	urllib.parse	urllib 일부 urllib 일부 urllib 일부 urllib 일부 하나의 urllib 패키지로 모아서 모듈을 기능별로 나눴고, urllib 모듈은 기능에 따라 여러 모듈로 흩어졌습니다.
urllib.request	urllib2 대부분	
urllib.error	urllib2 예외 부분	
urllib.response		
urllib.robotparser	robotparser	
http.server	BaseHTTPServer	하나의 http 패키지로 모아 server와 client 모듈로 구분했습니다.
http.server	CGIHTTPServer	
http.server	SimpleHTTPServer	
http.client	httplib	
http.cookies	Cookie	하나의 http 패키지로 모았습니다.
http.cookiejar	cookielib	
html.parser	HTMLParser	하나의 html 패키지로 모았습니다.
html.entities	htmlentitydefs	

01. Django 웹 개념 및 개발환경

■ 웹 클라이언트 라이브러리

웹 클라이언트를 위한 파이썬 표준 라이브러리가 있지만, 실제 프로젝트에서는 외부 라이브러리인 `requests`, `beautifulsoup4` 등을 더 많이 사용하는 편입니다. 좀 더 간편하고 이해하기 쉬운 문법을 제공하기 때문

Python Shell 실행 파이썬 언어는 2가지 실행 방법을 제공합니다. 하나는 `python` 명령어로 파이썬 스크립트 파일 즉, `*.py` 파일을 실행하는 방법(> `python example.py`). 또 다른 방법은 파이썬 셸 모드에서 라인 단위로 실행하는 것입니다. 파이썬 셸 모드로 진입하려면 `python` 명령어만 입력(>`python`

01. Django 웹 개념 및 개발환경

■ 웹 클라이언트 라이브러리

2.2.1 urllib.parse 모듈

```
C:\WRedBook\ch2>python
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 18:41:36) [MSC v.1900 64 bit (AMD64)] on win32
Type 'help', 'copyright', 'credits' or 'license' for more information.
>>>
>>> from urllib.parse import urlparse
>>> result = urlparse('http://www.python.org:80/guido/python.html;philosophy?overall=3#n10')
>>> result
ParseResult(scheme='http', netloc='www.python.org:80',
path='/guido/python.html', params='philosophy', query='overall=3', fragment='n10')
```

scheme : URL에 사용된 프로토콜을 의미합니다.

- netloc : 네트워크 위치. user :password@host :port 형식으로 표현되며, HTTP 프로토콜인 경우는 host:port 형식.
- path : 파일이나 애플리케이션 경로를 의미.
- params : 애플리케이션에 전달될 매개변수입니다. 현재는 사용하지 않음.
- query : 질의 문자열 또는 매개변수로, 앰퍼샌드(&)로 구분된 이름=값 쌍 형식으로 표현.
- fragment : 문서 내의 앵커 등 조각을 지정.

01. Django 웹 개념 및 개발환경

■ 웹 클라이언트 라이브러리

```
urlopen(url, data=None, [timeout])
```

사용 케이스	사용 방법
URL로 GET/POST 방식의 간단한 요청 처리	urlopen() 함수만으로 가능
PUT, HEAD 메소드 등, 헤더 조작이 필요한 경우	Request 클래스를 같이 사용
인증, 쿠키, 프록시 등 복잡한 요청 처리	인증/쿠키/프록시 해당 핸들러 클래스를 같이 사용

표 2-2 urlopen() 함수 사용 방법

- url 인자로 지정한 URL로 연결하고, 유사 파일 객체를 반환.
url 인자는 문자열이거나, Request 클래스의 인스턴스가 올 수 있음.
- url에 file 스킴을 지정하면 로컬 파일을 열 수 있음.
- 디폴트 요청 방식은 GET이고, 웹 서버에 전달할 파라미터가 있으면 질의 문자열을 url 인자에 포함해서 보냄.
- 요청 방식을 POST로 보내고 싶으면 data 인자에 질의 문자열을 지정해주면 됨.
- 옵션인 timeout은 응답을 기다리는 타임아웃 시간을 초로 표시

01. Django 웹 개념 및 개발환경

■ 웹 클라이언트 라이브러리

2.2.3 urllib.request 모듈 예제

```

C:\Users\Wshkim>cd C:\WRedBook\ch2
C:\WRedBook\ch2>notepad parse_image.py

from urllib.request import urlopen
from html.parser import HTMLParser

class ImageParser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        if tag != 'img':
            return
        if not hasattr(self, 'result'):
            self.result = []
        for name, value in attrs:
            if name == 'src':
                self.result.append(value)

def parse_image(data):
    parser = ImageParser()
    parser.feed(data)
    dataSet = set(x for x in parser.result)
    return dataSet

def main():
    url = "http://www.google.co.kr"

    with urlopen(url) as f:
        charset = f.info().get_param('charset')
        data = f.read().decode(charset)

    dataSet = parse_image(data)

    print("\n>>>>>>>>>> Fetch Images from", url)
    print("\n".join(sorted(dataSet)))

if __name__ == '__main__':
    main()

```

01. Django 웹 개념 및 개발환경

■ 웹 클라이언트 라이브러리

2.2.4 http.client 모듈

순번	코딩 순서	코딩 예시
1	연결 객체 생성	<code>conn = http.client.HTTPConnection("www.python.org")</code>
2	요청을 보냄	<code>conn.request("GET", "/index.html")</code>
3	응답 객체 생성	<code>response = conn.getresponse()</code>
4	응답 데이터를 읽음	<code>data = response.read()</code>
5	연결을 닫음	<code>conn.close()</code>

표 2-3 http.client 모듈 사용 시 코딩 순서

GET, POST 이외의 방식으로 요청을 보내거나, 요청 헤더와 바디 사이에 타이머를 두어 시간을 지연시키는 등 `urllib.request` 모듈로는 쉽게 처리할 수 없는 경우 혹은 HTTP 프로토콜 요청에 대한 저수준의 더 세밀한 기능이 필요할 때는 `http.client` 모듈을 사용함

01. Django 웹 개념 및 개발환경

■ 웹 클라이언트 라이브러리

2.2.5 http.client 모듈 예제

예제 2-12 http_client 모듈 예제

소스제공: ch2Wdownload_image.py

```
C:\Users\Wshkim>cd C:\WRedBook\ch2
C:\WRedBook\ch2>notepad download_image.py
```

```
import os
from http.client import HTTPConnection
from urllib.parse import urljoin, urlunparse
from urllib.request import urlopen
from html.parser import HTMLParser
```

```
class ImageParser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        if tag != 'img':
            return
        if not hasattr(self, 'result'):
            self.result = []
        for name, value in attrs:
            if name == 'src':
                self.result.append(value)
```

```
def download_image(url, data):
    if not os.path.exists('DOWNLOAD'):
        os.makedirs('DOWNLOAD')
```

```
parser = ImageParser()
parser.feed(data)
dataSet = set(x for x in parser.result)
```

```
for x in sorted(dataSet) :  
    imageUrl = urljoin(url, x)  
    basename = os.path.basename(imageUrl)  
    targetFile = os.path.join('DOWNLOAD', basename)  
  
    print("Downloading...", imageUrl)  
    urlretrieve(imageUrl, targetFile)  
  
def main():  
    host = "www.google.co.kr"  
  
    conn = HTTPConnection(host)  
    conn.request("GET", '')  
    resp = conn.getresponse()  
  
    charset = resp.msg.get_param('charset')  
    data = resp.read().decode(charset)  
    conn.close()  
  
    print("\n>>>>>>> Download Images from", host)  
    url = urlunparse(('http', host, '', '', '', ''))  
    download_image(url, data)  
  
if __name__ == '__main__':  
    main()
```

01. Django 웹 개념 및 개발환경

■ 웹 서버 라이브러리

2.3.1 간단한 웹 서버

예제 2-13 간단한 웹 서버

소스제공: ch2\my_httpserver.py

```
C:\Users\wshkim>cd C:\wRedBook\ch2
C:\wRedBook\ch2>notepad my_httpserver.py

from http.server import HTTPServer, BaseHTTPRequestHandler ①

class MyHandler(BaseHTTPRequestHandler): ②
    def do_GET(self):
        self.send_response_only(200, 'OK')
        self.send_header('Content-Type', 'text/plain')
        self.end_headers()
        self.wfile.write(b"Hello World")

if __name__ == '__main__': ③
    server = HTTPServer(('', 8888), MyHandler)
    print("Started WebServer on port 8888...")
    print("Press ^C to quit WebServer.")
    server.serve_forever() ④
```

클래스명	주요 기능
HTTPServer	• 웹 서버를 만들기 위한 클래스로, 서버 IP와 PORT를 바인딩함 • HTTPServer 객체 생성 시, 핸들러 클래스가 반드시 필요함
BaseHTTPRequestHandler	• 핸들러를 만들기 위한 기반 클래스로, HTTP 프로토콜 처리 로직이 들어있음 • 이 클래스를 상속받아, 자신의 로직 처리를 담당하는 핸들러 클래스를 만듦
SimpleHTTPRequestHandler	• BaseHTTPRequestHandler 클래스를 상속받아 만든 클래스 • GET과 HEAD 메소드 처리가 가능한 핸들러 클래스
CGIHTTPRequestHandler	• SimpleHTTPRequestHandler 클래스를 상속받아 만든 클래스 • 추가적으로 POST 메소드와 CGI 처리가 가능한 핸들러 클래스

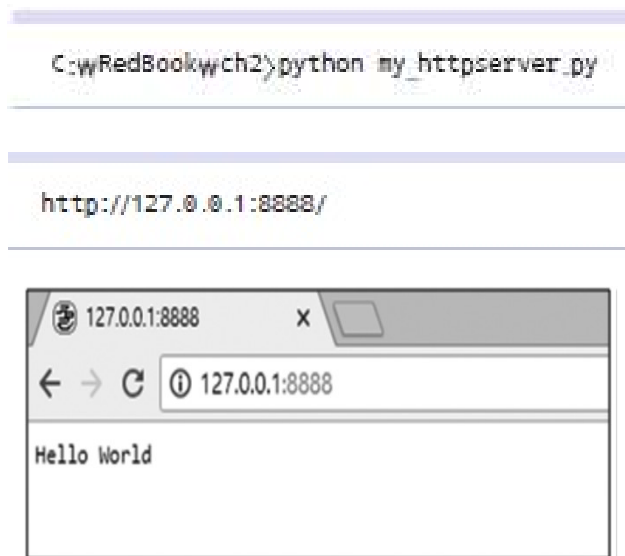
01. Django 웹 개념 및 개발환경

■ 웹 서버 라이브러리

2.3.2 HTTPServer 및 BaseHTTPRequestHandler 클래스

앞 절에서 설명한 것처럼 우리가 원하는 웹 서버를 만들기 위해서는 기반 클래스를 임포트하거나 상속받아야 함.

이처럼 기반이 되는 클래스가 바로 HTTPServer 및 BaseHTTPRequestHandler 클래스



```
C:\wRedBook\ch2>python my_httpserver.py
Started WebServer on port 8888...
Press ^C to quit WebServer.
```

그림 2-5 웹 서버(my_httpserver.py) 접속 결과

그림 2-4 웹 서버(my_httpserver.py) 실행 화면

01. Django 웹 개념 및 개발환경

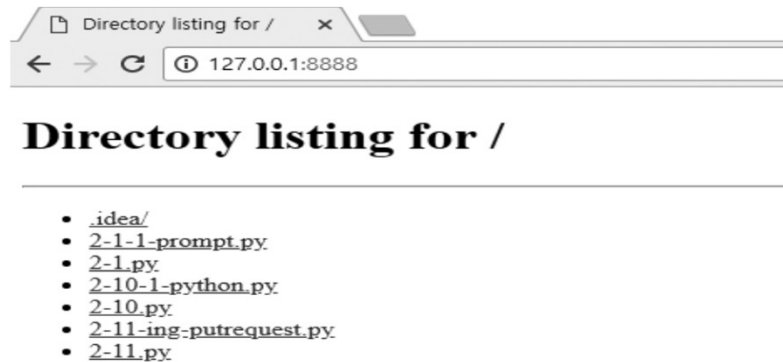
■ 웹 서버 라이브러리

2.3.3 SimpleHTTPRequestHandler

```
C:\RedBook\ch2>python -m http.server 8888
```

```
C:\RedBook\ch2>python -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
```

그림 2-6 Simple 웹 서버 실행 화면



디렉토리 리스트가 나오는 것은 SimpleHTTPRequestHandler의 do_GET() 메소드가 디렉토리 리스트를 반환하도록 구현되어 있기 때문

01. Django 웹 개념 및 개발환경

■ 웹 서버 라이브러리

2.3.4 CGIHTTPRequestHandler 클래스

SimpleHTTPRequestHandler 클래스와 유사하게 CGIHTTPRequestHandler 클래스가 미리 구현되어 있어서 필요할 때 즉시 웹 서버를 실행할 수 있습니다. CGIHTTPRequestHandler 클래스에는 do_POST() 메소드가 정의되어 있어서 POST 방식을 처리할 수 있습니다. 물론 SimpleHTTPRequestHandler 클래스를 상속받고 있어서, GET 및 HEAD 방식도 처리합니다.

```
C:\Users\wshkim>cd C:\RedBook\ch2\cgi-server  
C:\RedBook\ch2\cgi-server>python -m http.server 8888 --cgi
```

```
C:\RedBook\ch2>cd cgi-server  
C:\RedBook\ch2\cgi-server>python -m http.server 8888 --cgi  
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
```

그림 2-8 CGI 웹 서버 실행 화면

01. Django 웹 개념 및 개발환경

■ 웹 서버 라이브러리

2.3.5 xxxHTTPServer 모듈 간의 관계(파이썬 2.x 버전만 해당)

모든 HTTP 웹 서버는 BaseHTTPServer 모듈의 HTTPServer 클래스를 사용하여 작성하고, 웹 서버에 사용되는 핸들러는 BaseHTTPServer 모듈의 BaseHTTPRequestHandler를 상속받아 작성. BaseHTTPServer 모듈이 기본이 되고, 이를 확장한 모듈이 SimpleHTTPServer 모듈이며, CGIHTTPServer 모듈은 SimpleHTTPServer 모듈을 확장하여 작성

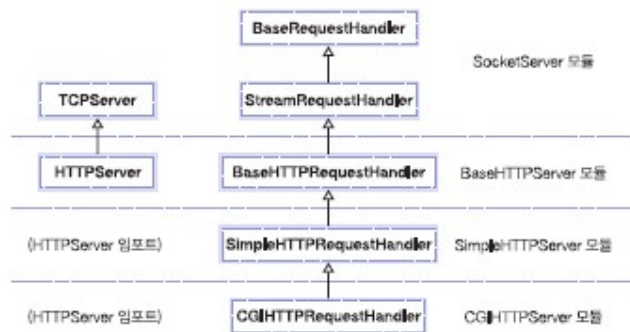


그림 2-11 웹 서버용 모듈의 test() 함수 간 호출 관계

그림 2-10 xxxHTTPServer 모듈의 클래스 간 상속도

01. Django 웹 개념 및 개발환경

■ CGI/WSGI 라이브러리

2.4.1 CGI 관련 모듈

웹 서버가 사용자의 요청을 애플리케이션에 전달하고 애플리케이션의 처리 결과를 애플리케이션으로부터 되돌려받기 위한, 즉 웹 서버와 애플리케이션 간에 데이터를 주고받기 위한 규격을 **CGICommon Gateway Interface**라고 함

01. Django 웹 개념 및 개발환경

■ CGI/WSGI 라이브러리

2.4.2 WSGI 개요

앞 절에서 설명한 CGI 방식은 요청이 들어올 때마다 처리를 위한 프로세스가 생성되는 방식이라 서, 짧은 시간에 수천, 수만의 다량 요청을 받으면 서버의 부하가 높아져서 프로세스가 멈추거나 다운될 수도 있습니다. 이러한 CGI의 단점을 해결하고, 파이썬 언어로 애플리케이션을 좀 더 쉽게 작성할 수 있도록 웹 서버와 웹 애플리케이션 간에 연동 규격을 정의한 것이 WSGI 규격

01. Django 웹 개념 및 개발환경

■ CGI/WSGI 라이브러리

2.4.3 WSGI 서버의 애플리케이션 처리 과정

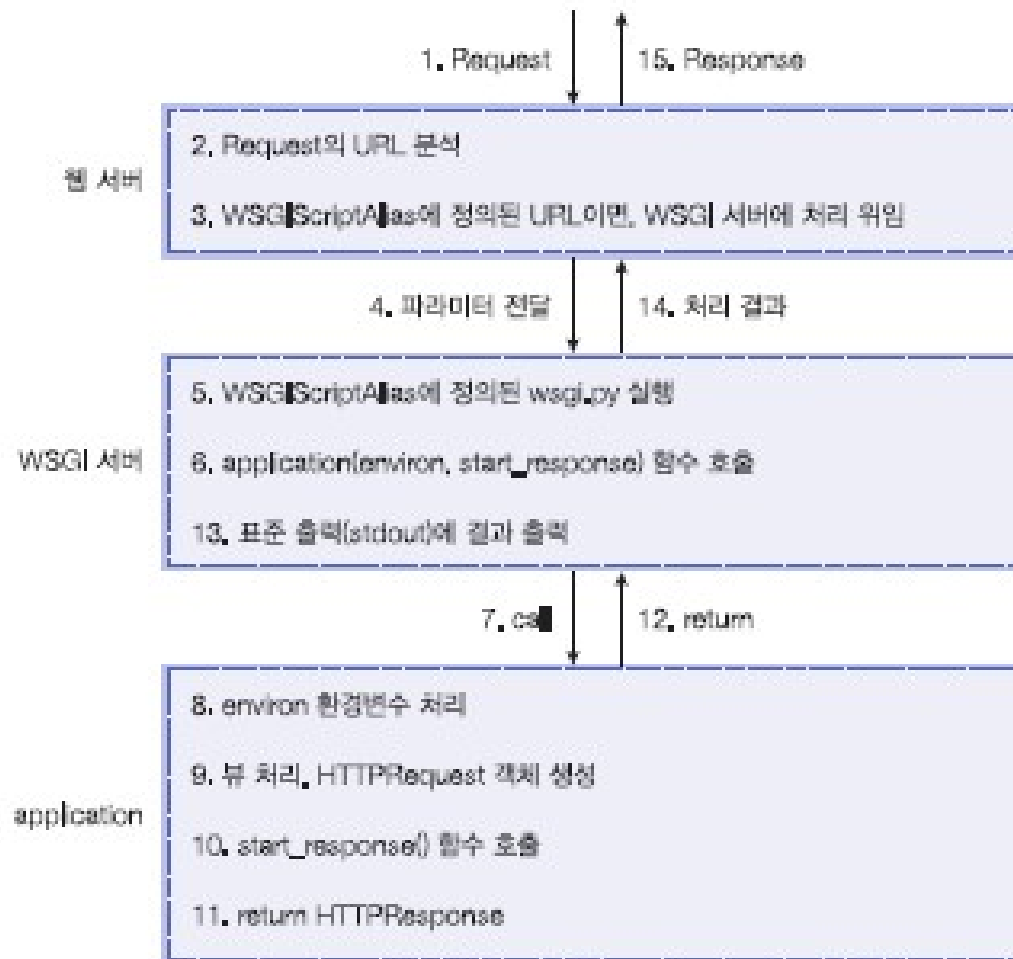


그림 2-12 WSGI 애플리케이션의 처리 순서

01. Django 웹 개념 및 개발환경

■ CGI/WSGI 라이브러리

2.4.4 wsgiref.simple_server 모듈

이 모듈은 WSGI 스펙을 준수하는 웹 서버(일명, WSGI 서버)에 대한 참조 reference 서버, 즉 개발자에게 참고가 될 수 있도록 미리 만들어 놓은 WSGIServer 클래스와 WSGIRequestHandler 클래스를 정의.

장고의 runserver도 이들 클래스를 사용하여 만든 테스트용 웹 서버

예제 2-16 WSGI 서버

소스제공: ch2\Wwsgi-server\my_wsgiserver.py

```
C:\Users\wshkim>cd C:\wRedBook\ch2\wwsgi-server
C:\wRedBook\ch2\wwsgi-server>notepad my_wsgiserver.py

from wsgiref.simple_server import make_server

def my_app(environ, start_response):

    status = '200 OK'
    headers = [('Content-Type', 'text/plain')]
    start_response(status, headers)

    response = [b"This is a sample WSGI Application."]

    return response

if __name__ == '__main__':
    print("Started WSGI Server on port 8888...")
    server = make_server('', 8888, my_app)
    server.serve_forever()
```

01. Django 웹 개념 및 개발환경

■ CGI/WSGI 라이브러리

2.4.5 WSGI 서버 동작 확인

```
C:\Users\shkim>cd C:\RedBook\ch2\wsgi-server  
C:\RedBook\ch2\wsgi-server>python my_wsgiserver.py
```

```
C:\RedBook\ch2\wsgi-server>python my_wsgiserver.py  
Started WSGI Server on port 8888...
```

그림 2-14 WSGI 서버(my_wsgiserver.py) 실행 화면



그림 2-15 WSGI 서버에서 my_app() Application 실행 결과

01. Django 웹 개념 및 개발환경

■ Django 개발환경

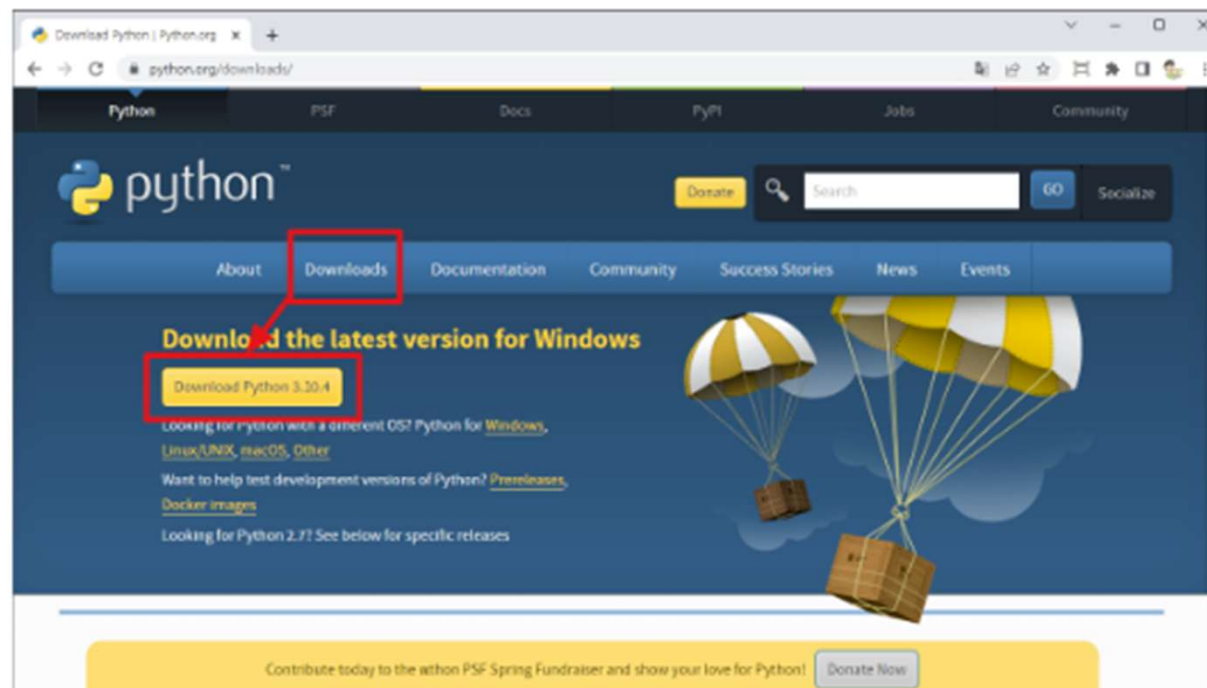
- 파이썬 설치
- Django 설치
- PyCharm 설치

01. Django 웹 개념 및 개발환경

■ Django 개발환경

■ 파이썬 설치

- 1) 파이썬 공식 홈페이지 주소: www.python.org
- 2) 파이썬 공식 홈페이지에서 [Downloads] 메뉴를 누르고 윈도우용 파이썬 설치 파일을 내려받기

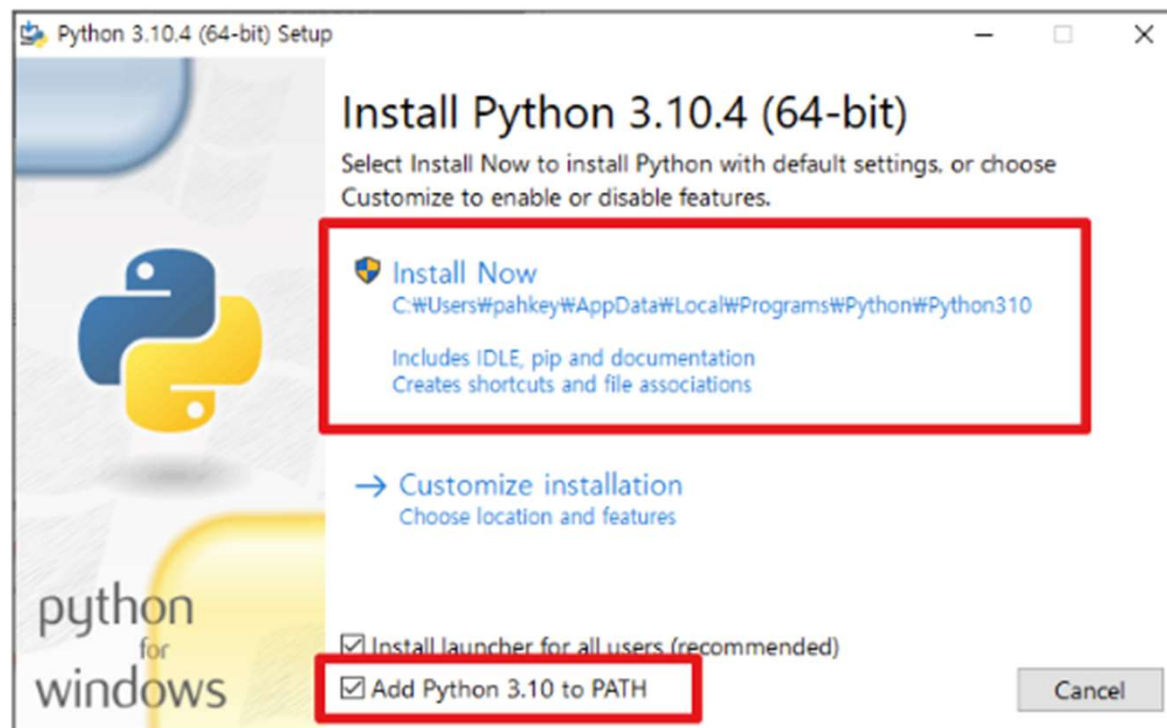


01. Django 웹 개념 및 개발환경

■ Django 개발환경

■ 파이썬 설치

3) 파이썬 설치 화면이 열리면 본격적으로 설치 작업을 하기 전에 반드시 아래쪽에 있는 <Add Python 3.10 to PATH> 옵션을 선택. 그 다음 <Install Now>를 선택하면 바로 설치가 진행된다. 설치를 완료하면 <close>를 눌러 설치 프로그램을 종료한다.



01. Django 웹 개념 및 개발환경

■ Django 개발환경

- 파이썬 설치

4) 파이썬이 제대로 설치됐는지 확인하기 위해 명령 프롬프트를 열기. 윈도우 작업 표시줄에 있는 검색란에 cmd를 입력 후 <Enter> 를 누르자. 명령 프롬프트가 열리면 다음 명령을 입력해 자신의 컴퓨터에 설치된 파이썬 버전을 확인해 보자. 오류 없이 파이썬 버전이 제대로 출력되면 설치에 성공한 것이다.

[명령 프롬프트]

```
C:\Users\pahkey> python -V  
Python 3.10.4
```

01. Django 웹 개념 및 개발환경

■ Django 개발환경

- Django 설치

- 1) 장고를 파이썬 가상 환경에 설치할 것이다. 파이썬 가상 환경은 파이썬 프로젝트를 진행할 때 독립된 환경을 만들어 주는 도구다.
- 2) 윈도우에서 명령 프롬프트를 실행하고 다음 명령어를 입력해 C:/venvs라는 디렉터리를 만들자. 루트 디렉터리를 반드시 C:/venvs로 해야 하는 것은 아니지만 실습 편의를 위해 이대로 지정하자.

```
C:\Users\pahkey> cd \  
C:\> mkdir venvs  
C:\> cd venvs
```

01. Django 웹 개념 및 개발환경

■ Django 개발환경

- Django 설치

3) 파이썬 가상 환경을 만든 다음 명령어를 입력해 실행하자.

```
C:\venvs> python -m venv mysite
```

4) 가상 환경에 진입하려면 우리가 생성한 mysite 가상 환경에 있는 Scripts 디렉터리의 activate 명령을 수행해야 한다. 다음 명령을 입력하여 mysite 가상 환경에 진입해 보자.

```
C:\venvs>cd C:\venvs\mysite\Scripts
C:\venvs\mysite\Scripts> activate
(mysite) C:\venvs\mysite\Scripts>
```

01. Django 웹 개념 및 개발환경

■ Django 개발환경

- Django 설치

5) C:/ 왼쪽에 (mysite)라는 프롬프트를 확인할 수 있다. 이름에서 볼 수 있듯 현재 여러분이 진입한 가상 환경을 의미한다.

6) 명령 프롬프트 왼쪽에 (mysite) 프롬프트가 보이는지 확인하자. 만약 명령 프롬프트 왼쪽에 (mysite) 프롬프트가 보이지 않는다면 바로 이전의 실습을 참고하여 가상 환경에 진입한 상태에서 장고 설치를 진행하자.

```
C:\venvs\mysite\Scripts> activate  
(mysite) C:\venvs\mysite\Scripts>
```

01. Django 웹 개념 및 개발환경

■ Django 개발환경

- Django 설치

7) mysite 가상 환경에 진입한 상태에서 `pip install django==4.0.3` 명령을 입력하자. `pip`은 파이썬 라이브러리를 설치하고 관리해 주는 파이썬 도구이다. `pip install django==4.0.3`는 `pip`으로 장고 4.0.3 버전을 설치하는 명령이라고 생각하면 된다. 다음 화면이 나오면 장고 설치가 잘 된 것이다.

```
(mysite) C:\venvs\mysite\Scripts> pip install django==4.0.3
Collecting django
  Using cached https://files.pythonhosted.org/packages/01/a5/fb3dad18422fcd4241d18460a1fe17542bfdeadcf74e3861d1a2dfc9e459/Django-4.0.3-py3-none-any.whl
(... 생략 ...)
You should consider upgrading via the 'python -m pip install --upgrade pip' command.
```

01. Django 웹 개념 및 개발환경

■ Django 개발환경

- Django 설치

8) 마지막으로 경고(WARNING) 문구가 보인다면 pip이 최신 버전이 아니라는 내용이다. 경고 메시지에 따라 `python -m pip install --upgrade pip` 명령을 입력해 pip을 최신 버전으로 설치하자.

```
(mysite) C:\venvs\mysite\Scripts> python -m pip install --upgrade pip
Collecting pip
  Downloading https://files.pythonhosted.org/packages/54/0c/d01aa759fdc501a58f431eb594a17495f15b88da142ce14b5845662c13f3/pip-
(... 생략 ...)
Successfully installed pip-20.0.2
```

02

블로그 어플리케이션 만들기

02. 블로그 어플리케이션 만들기

■ Django

- 가상환경 활성화시키기

```
.\venv\Scripts\activate
```

```
(venv) PS C:\Users\SYPark\PycharmProjects\pythonProject2>
```

- 장고 설치하기

```
pip install Django~=4.1.0
```

02. 블로그 어플리케이션 만들기

■ Django

■ 장고 설치 결과

```
Collecting Django~=4.1.0
  Using cached Django-4.1.13-py3-none-any.whl (8.1 MB)
Collecting sqlparse>=0.2.2
  Using cached sqlparse-0.5.0-py3-none-any.whl (43 kB)
Collecting asgiref<4,>=3.5.2
  Using cached asgiref-3.8.1-py3-none-any.whl (23 kB)
Collecting tzdata
  Using cached tzdata-2024.1-py2.py3-none-any.whl (345 kB)
Collecting typing-extensions>=4
  Using cached typing_extensions-4.12.2-py3-none-any.whl (37 kB)
Installing collected packages: typing-extensions, tzdata, sqlparse, asgiref, Django
Successfully installed Django-4.1.13 asgiref-3.8.1 sqlparse-0.5.0 typing-extensions-4.12.2 tzdata-2024.1
WARNING: You are using pip version 21.1.2; however, version 24.1.2 is available.
You should consider upgrading via the 'C:\Users\SYPark\PycharmProjects\pythonProject2\venv\Scripts\python.exe -m pip install --upgrade pip' command.
```

02. 블로그 어플리케이션 만들기

■ Django

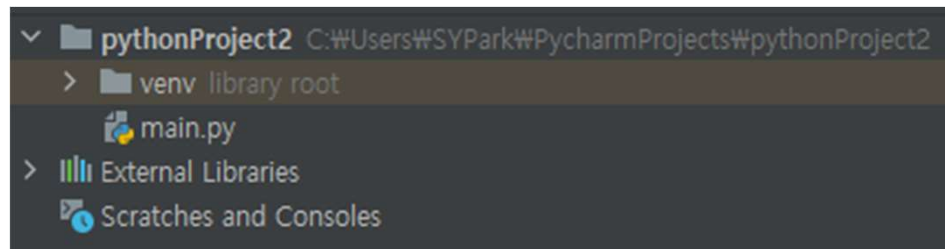
- 장고 설치 확인

```
(venv) PS C:\Users\SYPark\PycharmProjects\pythonProject2> python -m django --version  
4.1.13
```

02. 블로그 어플리케이션 만들기

■ Django

- 첫번째 프로젝트 만들기



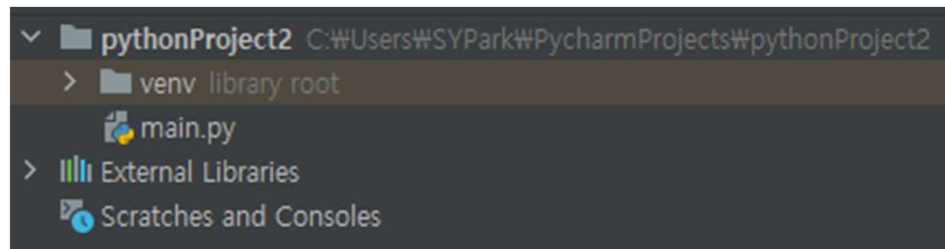
- 초기 프로젝트 파일 구조 생성 명령어

```
django-admin startproject mysite
```

02. 블로그 어플리케이션 만들기

■ Django

- 첫번째 프로젝트 만들기



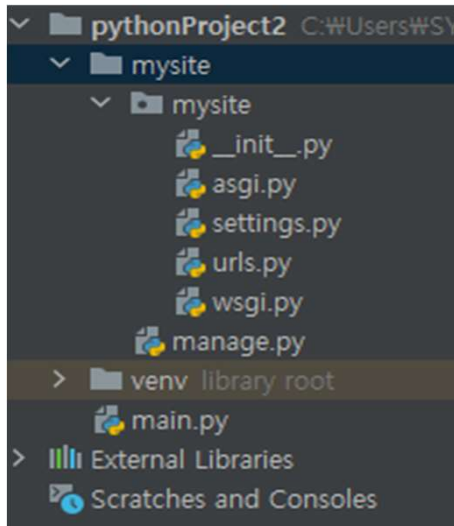
- 초기 프로젝트 파일 구조 생성 명령어

```
django-admin startproject mysite
```

02. 블로그 어플리케이션 만들기

■ Django

- 초기 프로젝트 파일 구조 생성 결과



manage.py : 프로젝트와 상호작용하는데 사용되는 커맨드라인 도구이다.

Mysite/ : 아래 파일들로 구성된 파이썬 패키지이다.

__init__.py : mysite 디렉터리를 파이썬 모듈로 취급하도록 파일썬에 지시하는 빈 파일이다.

Asgi.py : ASGI 호환 웹 서버와 함께 ASGI 애플리케이션으로 프로젝트를 실행하기 위한 구성이다.

Settings.py : 프로젝트 설정 및 구성을 나타내며 초기 기본 설정을 포함한다.

Urls.py : URL 패턴이 있는곳으로 정의된 각 URL은 뷰에 맵핑된다.

wsgi.py : WSGI 호환 웹서버와 함께 WSGI 애플리케이션 으로 프로젝트를 실행하기 위한 구성이다.

02. 블로그 어플리케이션 만들기

■ Django

- 초기 데이터베이스 마이그레이션 적용하기

```
cd mysite
```

```
python manage.py migrate
```

02. 블로그 어플리케이션 만들기

■ Django

- 초기 데이터베이스 마이그레이션 적용 결과 - 1

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```


02. 블로그 어플리케이션 만들기

■ Django

- 개발 서버 실행하기

```
python manage.py runserver
```

```
Watching for file changes with StatReloader
Performing system checks...

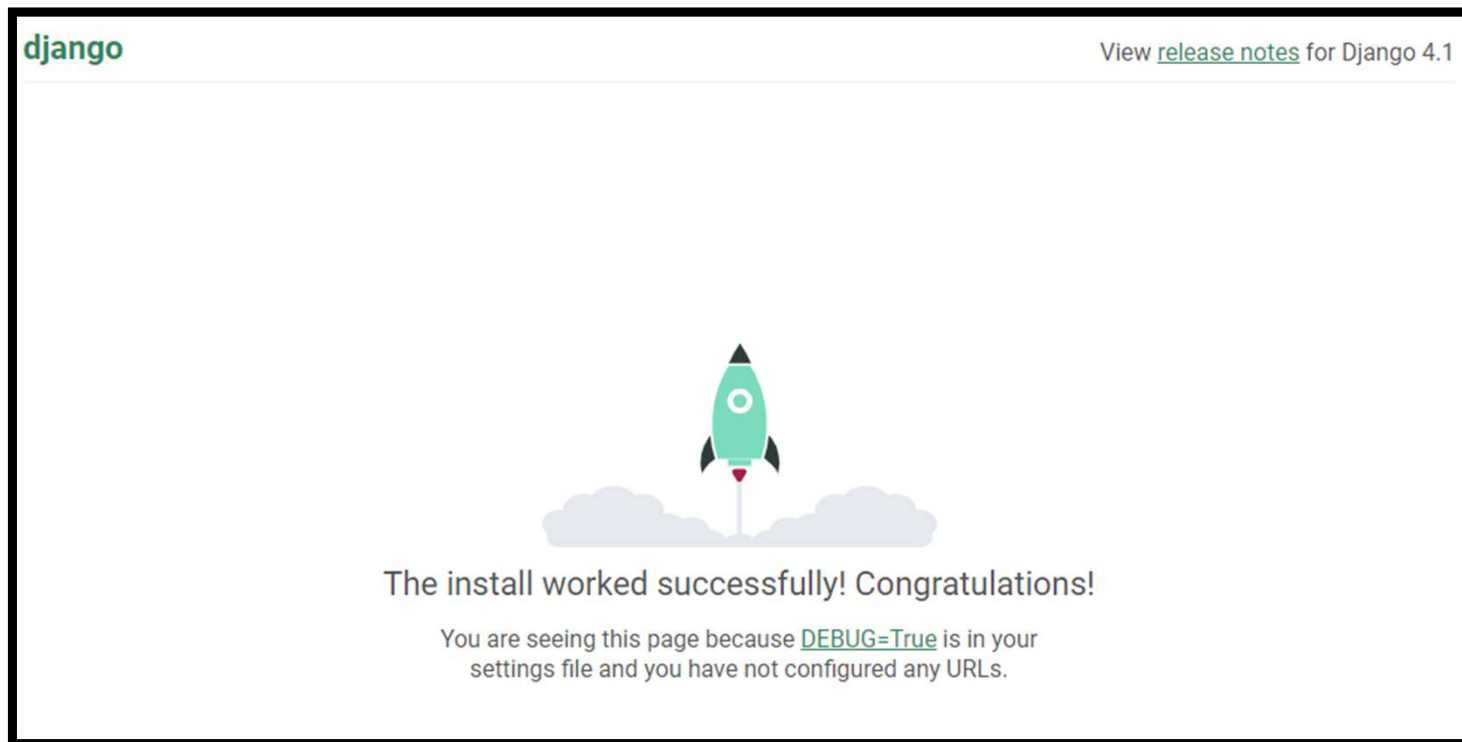
System check identified no issues (0 silenced).
July 09, 2024 - 04:27:23
Django version 4.1.13, using settings 'mysite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

02. 블로그 어플리케이션 만들기

■ Django

- 개발 서버 실행하기

<http://127.0.0.1:8000/>



02. 블로그 어플리케이션 만들기

■ Django

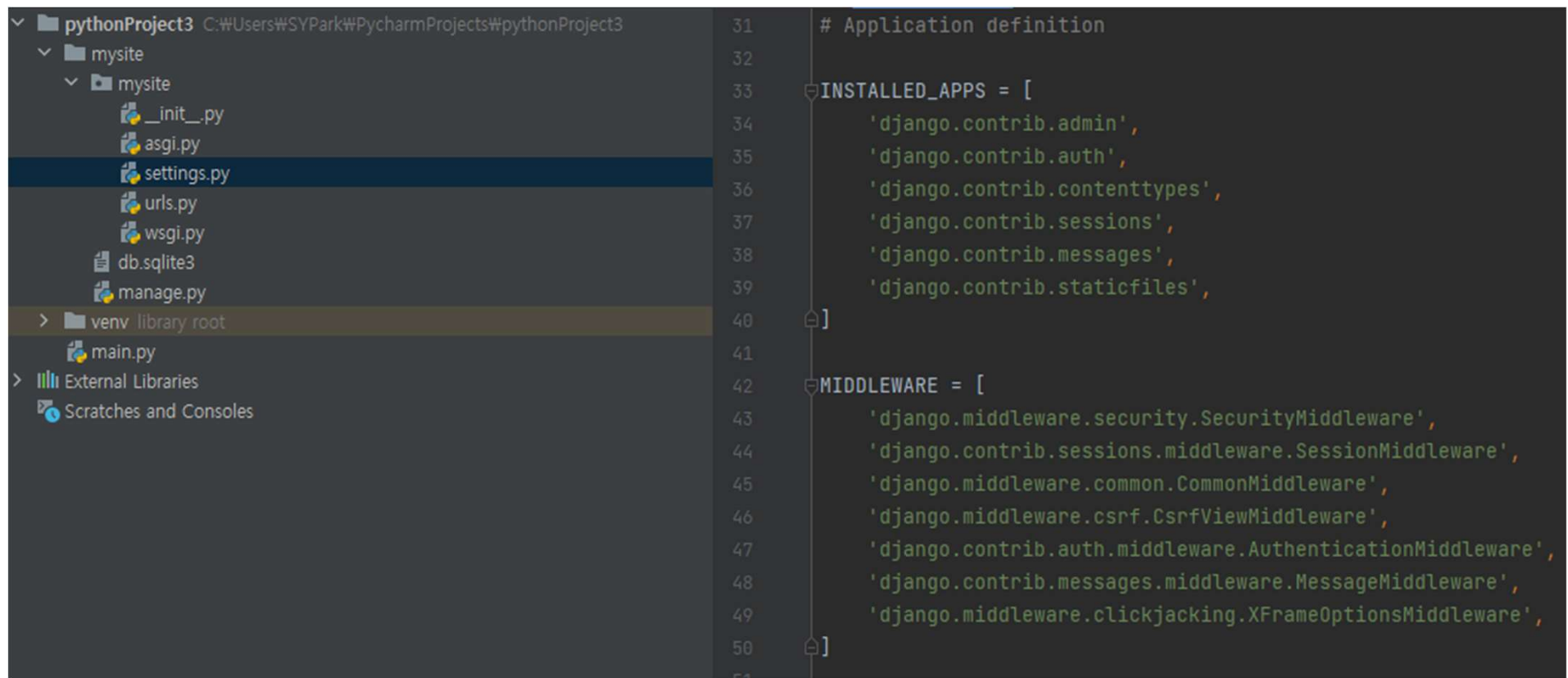
- 개발 서버 실행하기

```
[09/Jul/2024 04:28:48] "GET / HTTP/1.1" 200 10681
[09/Jul/2024 04:28:48] "GET /static/admin/css/fonts.css HTTP/1.1" 200 423
[09/Jul/2024 04:28:49] "GET /static/admin/fonts/Roboto-Bold-webfont.woff HTTP/1.1" 200 86184
[09/Jul/2024 04:28:49] "GET /static/admin/fonts/Roboto-Regular-webfont.woff HTTP/1.1" 200 85876
[09/Jul/2024 04:28:49] "GET /static/admin/fonts/Roboto-Light-webfont.woff HTTP/1.1" 200 85692
```

02. 블로그 어플리케이션 만들기

■ Django

- 프로젝트 설정하기



```
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40 ]
41
42 MIDDLEWARE = [
43     'django.middleware.security.SecurityMiddleware',
44     'django.contrib.sessions.middleware.SessionMiddleware',
45     'django.middleware.common.CommonMiddleware',
46     'django.middleware.csrf.CsrfViewMiddleware',
47     'django.contrib.auth.middleware.AuthenticationMiddleware',
48     'django.contrib.messages.middleware.MessageMiddleware',
49     'django.middleware.clickjacking.XFrameOptionsMiddleware',
50 ]
51
```

02. 블로그 어플리케이션 만들기

■ Django

■ 프로젝트 설정하기

DEBUG : 프로젝트의 디버그 모드를 켜고 끄는 부울값이다.

ALLOWED_HOSTS : 디버그 모드가 켜져 있거나 테스트가 동작 중일 때는 적용되지 않는다.

INSTALLED_APPS : 모든 프로젝트를 위해 편집해야하는 설정으로 장고에게 이 사이트에서 어떤 애플리케이션들을 활성화할지를 알려준다.

'django.contrib.admin', // 관리사이트

'django.contrib.auth', // 인증 프레임워크

'django.contrib.contenttypes', // 콘텐츠 유형을 처리하기 위한 프레임워크

'django.contrib.sessions', // 세션 프레임워크

'django.contrib.messages', // 메시징 프레임워크

'django.contrib.staticfiles', // 정적 파일 관리를 위한 프레임워크

MIDDLEWARE : 실행할 미들웨어가 포함된 목록이다.

ROOT_URLCONF : 애플리케이션의 루트 URL 패턴이 정의된 파이썬 모듈을 가리킨다.

DATABASES : 프로젝트에서 사용할 모든 데이터베이스 설정을 포함하는 딕셔너리이다.

02. 블로그 어플리케이션 만들기

■ Django

- 어플리케이션 생성하기

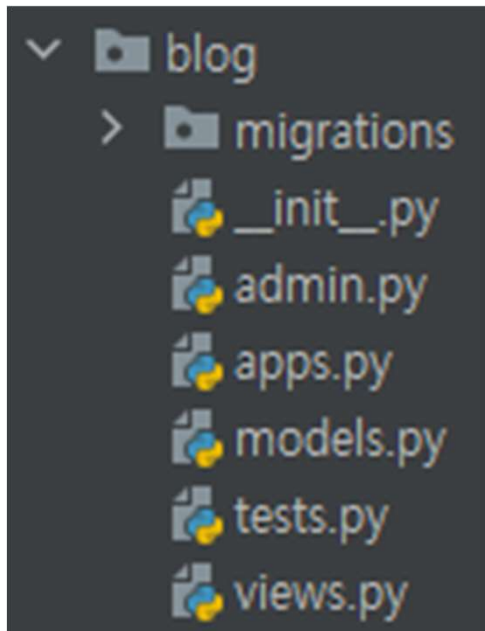
```
cd mysite
```

```
python manage.py startapp blog
```

02. 블로그 어플리케이션 만들기

■ Django

■ 어플리케이션 생성하기



`__init__.py` : 블로그 디렉토리를 파이썬 모듈로 취급하도록 파이썬에 지시하는 빈 파일이다.

`Admin.py` : 장고 관리 사이트에 포함할 모델을 등록하는 곳이다.

`Apps.py` : `blog` 애플리케이션의 주요 구성이 포함된다.

`Migrations` : 애플리케이션의 데이터베이스 마이그레이션이 포함된다.

`Models.py` : 애플리케이션의 데이터모델이 포함된다.

`Test.py` : 애플리케이션을 위한 테스트를 추가할 수 있다.

`View.py` : 애플리케이션의 로직이 위치한 곳이다. 각 뷰는 HTTP 요청을 수신해서 처리하고 응답을 반환한다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – models.py
- 게시물 모델 만들기

```
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()

    def __str__(self):
        return self.title
```

Title : 게시물의 제목 필드로 SQL 데이터베이스의 VARCHAR로 변환되는 CharField 필드이다.

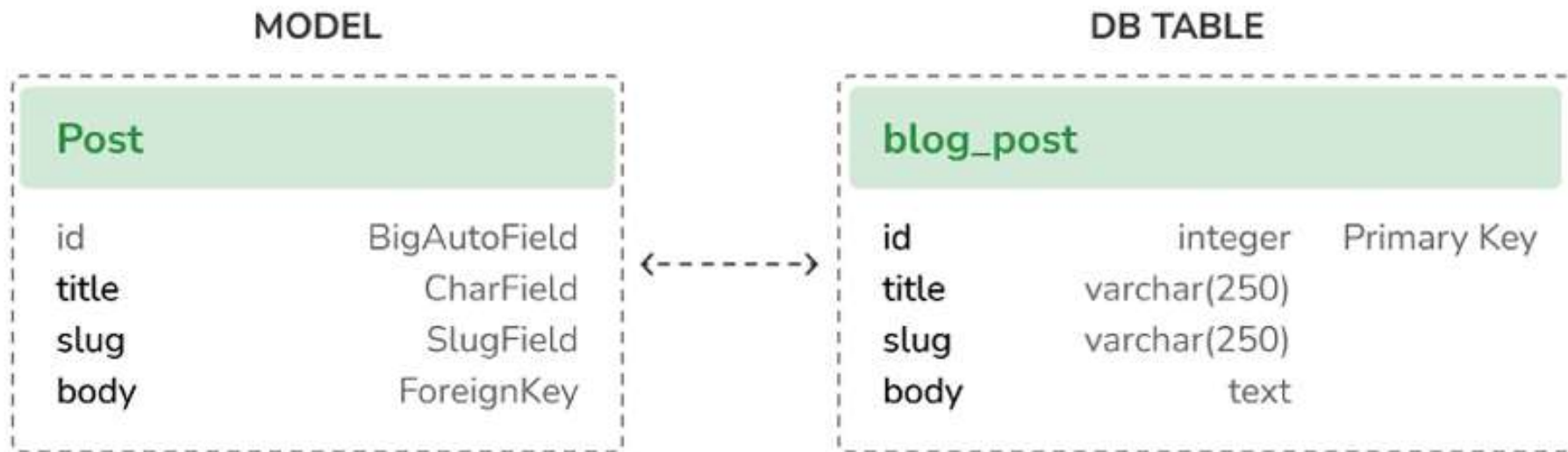
Slug : SQL 데이터베이스에서 VARCHAR로 변환되는 SlugField 필드이다.

Body : 게시물의 본문을 저장하는 필드이다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – models.py
- 게시물 모델 만들기
- 장고는 각 모델 필드(title,slug,body)에 맞는 데이터베이스 컬럼을 생성한다. 각 필드의 유형이 데이터베이스 데이터 유형에 어떻게 대응하는지 확인할 수 있다.



02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – models.py
- Datetime 필드 추가하기

```
from django.db import models
from django.utils import timezone

class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.title
```

Publish : SQL 데이터베이스의 DATETIME 컬럼으로 변환되는 DateTimeFiled 필드이다. 게시물이 게시된 날짜와 시간을 저장하는데 사용한다.

Created : DateTimeFiled로 게시물이 생성된 날짜와 시간을 저장하는데 사용한다.

Updated : DateTimeFiled로 게시물이 갱신된 마지막 날짜와 시간을 저장하는데 사용한다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – models.py
- 기본 정렬 순서 정의하기

```
from django.db import models
from django.utils import timezone

class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-publish']

    def __str__(self):
        return self.title
```

블로그 게시물은 일반적으로 시간의 역순으로 표시되기 때문에 모델 내부에 Meta 클래스를 추가하여 기본 정렬 순서를 정의한다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – models.py
- 데이터베이스 인덱스 추가하기

```
from django.db import models
from django.utils import timezone

class Post(models.Model):
    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-publish']
        indexes = [
            models.Index(fields=['-publish']),
        ]

    def __str__(self):
        return self.title
```

Publish 필드로 데이터베이스 인덱스를 정의한다. 기본값으로 publish 필드를 사용해서 결과를 정렬한다.

모델의 Meta 클래스에 indexes 옵션을 추가하여 모델의 데이터베이스 인덱스를 정의 한다.

이 인덱스는 오름차순 또는 내림차순으로 하나 이상의 필드로 구성하거나 함수 표현식 및 데이터베이스함수로 정의 할 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – settings.py
- 어플리케이션 활성화하기

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'blog.apps.BlogConfig'  
]
```

INSTALLED_APPS 설정에 blog.apps.BlogConfig를 추가하여 blog 애플리케이션을 활성화시킨다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – models.py
- 상태 필드 추가하기

```
from django.db import models
from django.utils import timezone

class Post(models.Model):

    class Status(models.TextChoices):
        DRAFT = 'DF', 'Draft'
        PUBLISHED = 'PB', 'Published'

    title = models.CharField(max_length=250)
    slug = models.SlugField(max_length=250)
    body = models.TextField()
    publish = models.DateTimeField(default=timezone.now)
    created = models.DateTimeField(auto_now_add=True)
    updated = models.DateTimeField(auto_now=True)
    status = models.CharField(max_length=2,
                              choices=Status.choices,
                              default=Status.DRAFT)

    class Meta:
        ordering = ['-publish']
        indexes = [
            models.Index(fields=['-publish']),
        ]

    def __str__(self):
        return self.title
```

블로그의 기능으로 게시할 준비가 될때
까지 게시물을 임시로 저장하는 기능이
있다.

블로그 게시물의 상태를 관리할 수 있는
상태 필드를 모델에 추가한다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – models.py
- 상태 필드 작동 확인하기

```
python manage.py shell
```

```
Python 3.10.6 (tags/v3.10.6:9c7b4bd, Aug 1 2022, 21:53:49) [MSC v.1932 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from blog.models import Post
>>> Post.Status.choices
```

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – models.py
- 상태 필드 작동 확인하기

```
python manage.py shell
```

```
>>> from blog.models import Post
>>> Post.Status.choices
[('DF', 'Draft'), ('PB', 'Published')]
>>> Post.Status.labels
['Draft', 'Published']
>>> Post.Status.values
['DF', 'PB']
>>> Post.Status.names
['DRAFT', 'PUBLISHED']
```

Choices시 값-레이블 쌍으로 열거형 항목들을 출력한다.

Post.Status.labels의 경우 사람이 읽을 수 있는 열거형 항목 명칭을 출력한다.

Post.Status.values의 경우 열거형 항목들의 값을 출력한다.

Post.Status.names의 경우 열거형 항목들의 명칭을 출력한다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기 – models.py
- 다 대 1 관계 추가하기

```
from django.contrib.auth.models import User
```

```
author = models.ForeignKey(User,  
                            on_delete=models.CASCADE,  
                            related_name='blog_posts')
```

어떤 사용자가 어떤 게시물을 작성했는지를 나타내는 사용자와 게시물 간의 관계를 생성한다. 장고는 사용자 계정을 처리하는 인증 프레임워크를 함께 제공한다.

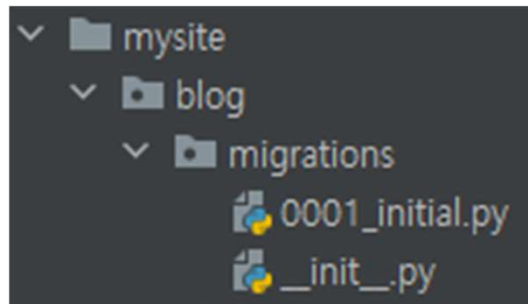
02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기
- 마이그레이션 생성 및 적용하기

```
python manage.py makemigrations blog
```

```
Migrations for 'blog':
  blog\migrations\0001_initial.py
    - Create model Post
    - Create index blog_post_publish_bb7600_idx on field(s) -publish of model post
```



블로그 게시물에 대한 데이터 모델을 만들었으므로 이를 통해 데이터베이스 테이블을 만드는 마이그레이션 작업을 진행한다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기
- 마이그레이션 생성 및 적용하기

```
python manage.py sqlmigrate blog 0001
```

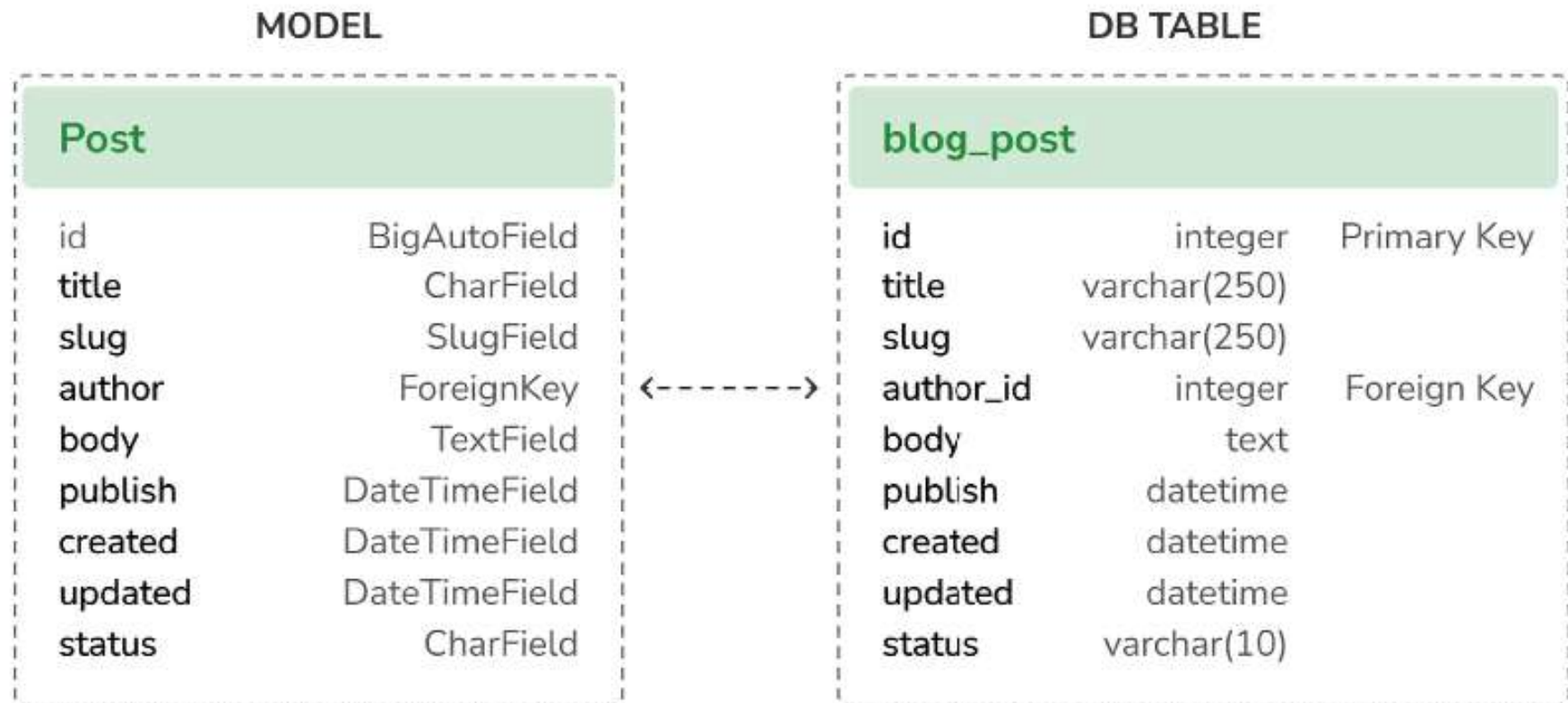
```
BEGIN;
--
-- Create model Post
--
CREATE TABLE "blog_post" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "title" varchar(250) NOT NULL, "slug" varchar(250) NOT NULL,
NOT NULL, "status" varchar(2) NOT NULL, "author_id" integer NOT NULL REFERENCES "auth_user" ("id") DEFERRABLE INITIALLY DEFERRED);
--
-- Create index blog_post_publish_bb7600_idx on field(s) -publish of model post
--
CREATE INDEX "blog_post_publish_bb7600_idx" ON "blog_post" ("publish" DESC);
CREATE INDEX "blog_post_slug_b95473f2" ON "blog_post" ("slug");
CREATE INDEX "blog_post_author_id_dd7a8485" ON "blog_post" ("author_id");
COMMIT;
```

장고가 모델의 테이블을 생성하기 위해 데이터베이스에서 실행할 sql 코드를 살펴보기 위해 sqlmigrate 명령어를 입력하여 SQL 질의어를 출력해본다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기
- 마이그레이션 생성 및 적용하기



최종적으로 마이그레이션된 형태이다. 모델의 필드와 데이터베이스간의 대응 여부를 확인할 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 블로그 데이터 모델 만들기
- 마이그레이션 생성 및 적용하기

```
python manage.py migrate
```

```
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, sessions
Running migrations:
  Applying blog.0001_initial... OK
```

최종적으로 데이터베이스를 새로운 모델과 동기화 진행하여 기존 마이그레이션을 적용하였다. 마이그레이션을 적용하고 나면 데이터베이스는 모델의 현재 상태를 반영한다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- 슈퍼유저 생성하기

```
python manage.py createsuperuser
```

```
Username (leave blank to use 'sympark'): admin
```

```
Email address: admin@admin.com
```

```
Password:
```

```
Password (again):
```

```
Superuser created successfully.
```

관리 사이트를 관리할 사용자를 생성한다. 이때, 가장 높은 권한을 가진 관리용 사용자를 생성한다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- 장고 관리 사이트

```
python manage.py runserver
```

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
July 09, 2024 - 05:22:05
Django version 4.1.13, using settings 'mysite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

개발 서버를 시작한다. 브라우저에서 <http://127.0.0.1:8000/admin/> 주소를 연다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- 장고 관리 사이트

A screenshot of the Django administration login interface. It features a dark teal header with the text "Django administration". Below the header, on a light blue background, are two input fields: "Username:" and "Password:". At the bottom center is a teal "Log in" button.

Django administration

Username:

Password:

Log in

앞에서 생성한 superuser의 아이디, 패스워드를 입력한다.
ID : admin / PW : 12345678

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- 장고 관리 사이트



Groups과 Users 모델은 Django.contrib.auth에 있는 장고 인증 프레임워크의 일부로 Users를 클릭하면 앞서 생성한 사용자를 볼 수 있다.

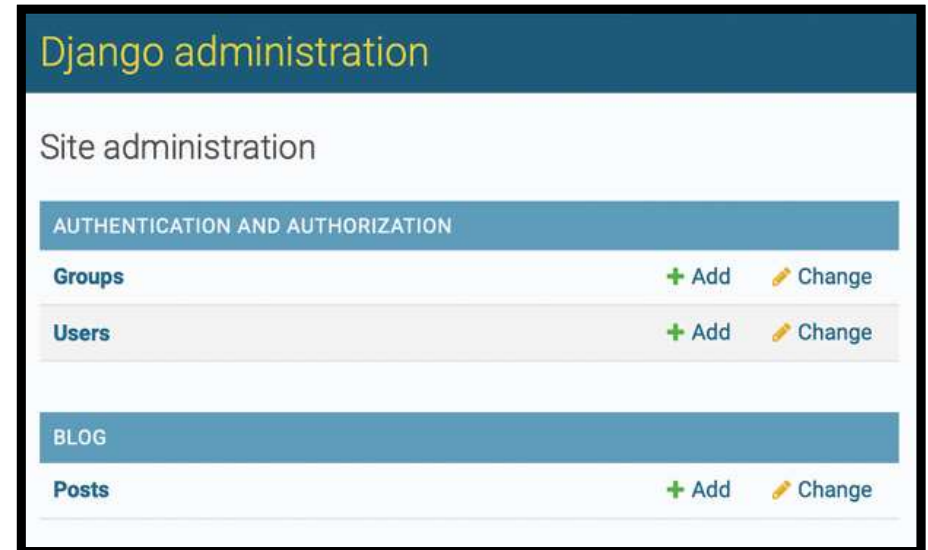
02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- 관리 사이트에 모델 추가하기 – admin.py

```
from django.contrib import admin
from .models import Post

admin.site.register(Post)
```



블로그 모델을 관리사이트에 추가하기, 저장 후 관리자 사이트를 재로드시 추가된 모습을 볼 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- 관리 사이트 이용하기

Add post

Title:

Slug:

Author:  

Body:

Publish: Date: Today 
Time: Now 
Note: You are 2 hours ahead of server time.

Status:

Add Post를 이용하여 게시글을 작성할 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 재실행시

```
.\venv\Scripts\activate
```

```
cd mysite
```

```
python manage.py runserver
```

접속 => <http://127.0.0.1:8000/admin/>

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- 모델 표시 방법 커스터마이징하기 전 모습

The post "Who was Django Reinhardt?" was added successfully.

Select post to change ADD POST +

Action: 0 of 1 selected

☐	POST
☐	Who was Django Reinhardt?

1 post

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- 모델 표시 방법 커스터마이징하기

```
from django.contrib import admin
from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ['title', 'slug', 'author', 'publish', 'status']
```

ModelAdmin을 상속한 커스텀 클래스를 사용해서 모델이 사이트에 등록되었음을 장고 관리 사이트에 알려준다.

이 클래스에서는 사이트에 모델을 표시하는 방법과 모델과 상호작용하는 방법에 관한 정보를 포함할 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- List_display 속성을 적용하여 모델 표시 방법 커스터마이징한 모습

Action:

Go

 0 of 1 selected

☐	TITLE	SLUG	AUTHOR	PUBLISH	STATUS
☐	asd	asd	admin	July 8, 2024, 8:30 p.m.	Draft

1 post

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- 모델 표시 방법 커스터마이징하기

```
from django.contrib import admin
from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ['title', 'slug', 'author', 'publish', 'status']
    list_filter = ['status', 'created', 'publish', 'author']
    search_fields = ['title', 'body']
    prepopulated_fields = {'slug': ('title',)}
    raw_id_fields = ['author']
    date_hierarchy = 'publish'
    ordering = ['status', 'publish']
```

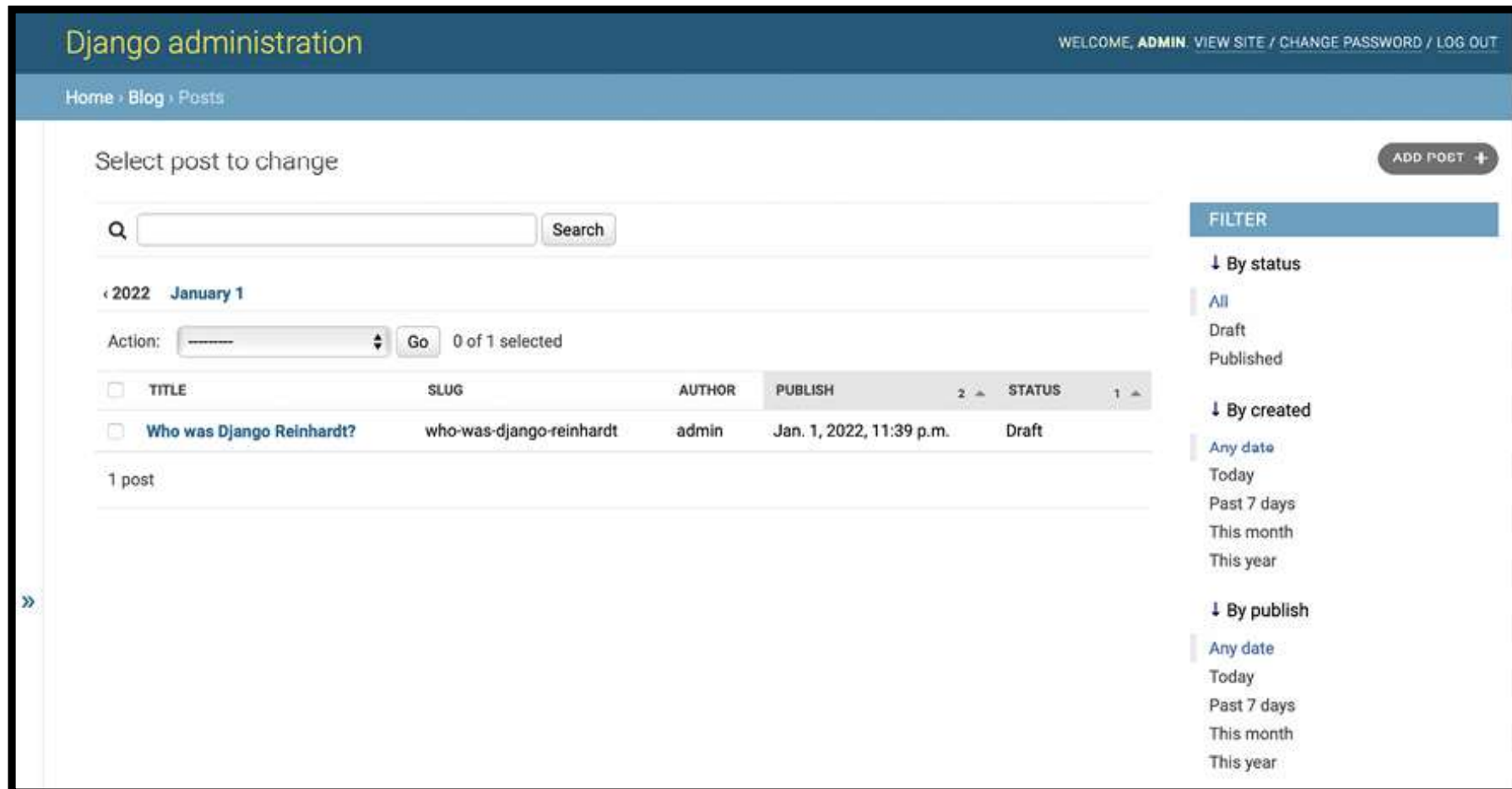
해당 ModelAdmin을 상속받은 PostAdmin 클래스를 이용하여 몇가지 추가 옵션으로 admin 모델을 커스터마이징 할 수 있다.

브라우저로 돌아가 게시물 목록 페이지를 새로고침하면 추가된 모습을 확인할 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

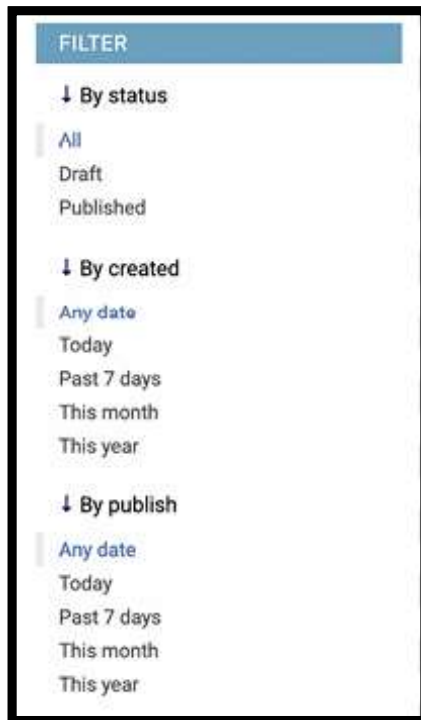
- 모델용 관리 사이트 만들기
- List_display 속성에 추가 옵션을 적용하여 모델 표시 방법 커스터마이징한 모습



02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- List_display 속성에 추가 옵션을 적용하여 모델 표시 방법 커스터마이징한 모습



List_filter 속성에 포함된 필드별로 결과를 필터링할 수 있는 오른쪽 사이드 바가 추가됨

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- List_display 속성에 추가 옵션을 적용하여 모델 표시 방법 커스터마이징한 모습



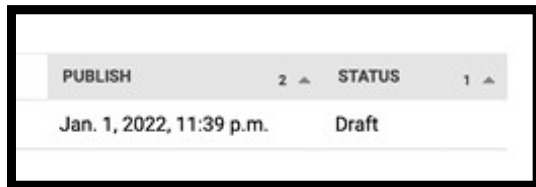
Search_fields 속성을 사용해서 검색 가능한 필드 목록을 정의하여 검색 바를 나타냄

또한, date_hierarchy 속성을 이용하여 검색 바 바로 아래에는 연월일 단계로 날짜를 고를 수 있는 내비게이션 링크를 위치시킴

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- List_display 속성에 추가 옵션을 적용하여 모델 표시 방법 커스터마이징한 모습



PUBLISH	2 ▲	STATUS	1 ▲
Jan. 1, 2022, 11:39 p.m.		Draft	

Ordering 속성을 사용하여 STATUS 및 PUBLISH 컬럼으로 정렬시킴

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- List_display 속성에 추가 옵션을 적용하여 모델 표시 방법 커스터마이징한 모습



The screenshot shows a web form titled "Add post". It contains two input fields. The first field is labeled "Title:" and contains the text "Who was Django Reinhardt?". The second field is labeled "Slug:" and contains the text "who-was-django-reinhardt". The slug field is automatically populated with a lowercase, hyphenated version of the title.

Prepopulated_fields 속성을 사용하여 title 필드의 입력으로 slug 필드를 채우도록 장고에 지정하였다. 오른쪽 상단에 ADD POSET 버튼을 클릭 후 게시물의 제목을 입력하면 SLUG 필드가 자동으로 채워지게 됨

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- List_display 속성에 추가 옵션을 적용하여 모델 표시 방법 커스터마이징한 모습



A screenshot of a Django model admin search widget. It consists of a rectangular box with a thick black border. Inside the box, on the left, is the label 'Author:' in bold black text. To the right of the label is a text input field containing the number '1'. To the right of the input field is a magnifying glass icon, representing a search button.

Raw_id_fields 속성을 사용하여 author 필드는 이제 조회 위젯과 함께 표시되며, 조회가 가능해진다.

몇 줄의 추가 옵션 코드로 모델이 관리 사이트에 표시되는 방식을 커스터마이징 가능하다. 장고 관리사이트를 커스터마이징하고 확장하는 방법은 장고에서 제공한다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기

블로그 게시물을 관리할 수 있는 어느 정도의 기능을 갖춘 관리 사이트이므로, 프로그래밍 방식으로 데이터베이스에서 콘텐츠를 읽고 쓰는 방법을 적용해보고자 한다.

이를 위해 장고에서는 장고 객체 관계형 매퍼(ORM)를 제공한다. ORM은 객체를 데이터베이스에 쉽게 생성, 검색, 갱신 및 삭제할 수 있는 강력한 데이터베이스 추상화 API이다.

ORM을 사용하면 파이썬의 객체지향 패러다임을 사용해서 SQL 쿼리를 생성할 수 있다.

이는 직접 SQL 쿼리를 작성하는 대신 파이썬 방식으로 데이터베이스와 상호작용할 수 있는 방법이다.

장고 ORM은 MySQL, PostgreSQL, SQLite, Oracle 및 MariaDB와 호환이 가능하다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기

장고의 ORM은 QuerySet을 기반으로 작동한다.

QuerySet은 데이터베이스에서 객체를 검색하기 위한 일련의 데이터베이스 쿼리 모음이다.

QuerySet에 필터를 적용해서 주어진 매개변수를 기반으로 쿼리 결과의 범위를 좁힐 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체만들기 => SQL INSERT

```
python manage.py shell
```

```
>>> from django.contrib.auth.models import User
>>> from blog.models import Post
>>> user = User.objects.get(username='admin')
>>> post = Post(title='Another post',slug='another-post',body='Post body.',author=user)
>>> post.save()
```

3행 `user = User.objects.get(username='admin')` 코드는 `get()` 메소드를 사용하여 데이터베이스에서 단일 객체를 검색한다. 이때, `username`의 `admin`이 존재하는지 존재하는지에 대한 여부를 확인 할 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체만들기 => SQL INSERT

```
python manage.py shell
```

```
>>> from django.contrib.auth.models import User
>>> from blog.models import Post
>>> user = User.objects.get(username='admin')
>>> post = Post(title='Another post',slug='another-post',body='Post body.',author=user)
>>> post.save()
```

4행 `post = Post(title='Another post',slug='another-post',body='Post body.',author=user)`
코드는 title, slug, body를 원하는 값으로 설정하고 앞서 조회해서 얻은 게시물의 저자인 user
를 author로 설정해서 Post 인스턴스를 생성한다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체만들기 => SQL INSERT

```
python manage.py shell
```

```
>>> from django.contrib.auth.models import User
>>> from blog.models import Post
>>> user = User.objects.get(username='admin')
>>> post = Post(title='Another post',slug='another-post',body='Post body.',author=user)
>>> post.save()
```

위 두 작업에 대하여 객체는 메모리에 존재하지만 아직 데이터베이스에 적용되지는 않는다. 실행 중에 사용할 수 있지만 데이터베이스에 저장되지 않은 파이썬 객체를 만든것이다. 최종적으로 위 2행의 코드를 데이터베이스에 적용하기 위해 5행의 `post.save()`를 적용해주어야 한다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체만들기 => SQL INSERT

```
python manage.py shell
```

```
>>> from django.contrib.auth.models import User
>>> from blog.models import Post
>>> user = User.objects.get(username='admin')
>>> post = Post(title='Another post', slug='another-post', body='Post body.', author=user)
>>> post.save()
```

결과적으로 해당 작업은 백그라운드에서 INSERT SQL을 수행한다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체만들기 => SQL INSERT

```
python manage.py runserver
```

◀ 2024 July 8 July 9					
Action:		Go	0 of 2 selected		
☐	TITLE	SLUG	AUTHOR	PUBLISH	STATUS
☐	asd	asd	admin	July 8, 2024, 8:30 p.m.	Draft
☐	Another post	another-post	admin	July 9, 2024, 6:38 p.m.	Draft
2 posts					

서버에 재접속시 데이터가 Insert된 모습을 확인 할 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체 수정하기 => SQL UPDATE

```
>>> from django.contrib.auth.models import User
>>> from blog.models import Post
>>> user = User.objects.get(username='admin')
>>> post = Post(title='Another post', slug='another-post', body='Post body.', author=user)
>>> post.title = 'New title'
>>> post.save()
```

게시물의 제목을 다른것으로 변경이 필요할 시 필드를 수정하고, 객체를 다시 저장한다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체 수정하기 => SQL UPDATE

◀ 2024 July 8 July 9					
Action: Go 0 of 3 selected					
☐	TITLE	SLUG	AUTHOR	PUBLISH	STATUS
☐	asd	asd	admin	July 8, 2024, 8:30 p.m.	Draft
☐	Another post	another-post	admin	July 9, 2024, 6:38 p.m.	Draft
☐	New title	another-post	admin	July 9, 2024, 6:52 p.m.	Draft

게시물의 제목이 New title로 변경된 것을 확인할 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체 조회하기 => SQL SELECT

```
>>> from django.contrib.auth.models import User
>>> from blog.models import Post
>>> user = User.objects.get(username='admin')
>>> post = Post(title='Another post',slug='another-post',body='Post body.',author=user)
>>> post.title = 'New title'
>>> post.save()
>>> all_post = Post.objects.all()
>>> Post.objects.all()
<QuerySet [<Post: New title>, <Post: New title>, <Post: Another post>, <Post: asd>]>
```

3행 Post에 objects.get()시에는 단일 객체를 조회 할때 사용한다.

7,8행 Post에 objects.all()시에는 테이블에서 모든 객체를 조회 할때 사용한다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체 조회하기 => SQL SELECT + Filter

```
>>> Post.objects.filter(publish__year=2024)
<QuerySet [<Post: New title>, <Post: New title>, <Post: Another post>, <Post: asd>]>
>>> Post.objects.filter(publish__year=2024,author__username='admin')
<QuerySet [<Post: New title>, <Post: New title>, <Post: Another post>, <Post: asd>]>
```

QuerySet을 필터링하기 위해서는 관리자의 filter() 메서드를 사용한다.
이때, QuerySet을 사용해서 2024년에 게시된 모든 게시물들을 조회할 수 있다.
또한, 사용자 이름이 admin이면서 2024년에 게시된 모든 게시물들을 조회할 수도 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체 조회하기 => SQL SELECT + Exclude

```
>>> Post.objects.filter(publish__year=2024)\  
... .exclude(title__startswith='Why')  
<QuerySet [<Post: New title>, <Post: New title>, <Post: Another post>, <Post: asd>]>
```

관리자의 `exclude()` 메서드를 사용해서 QuerySet에서 특정 결과를 제외할 수 있다.

예를 들어 제목이 Why로 시작하지 않는 2023년에 게시된 모든 게시물은 위와 같이 조회할 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체 조회하기 => SQL SELECT + Order_by

```
>>> Post.objects.order_by('title')
<QuerySet [<Post: Another post>, <Post: New title>, <Post: New title>, <Post: asd>]>
>>> Post.objects.order_by('-title')
<QuerySet [<Post: asd>, <Post: New title>, <Post: New title>, <Post: Another post>]>
```

관리자의 `order_by()` 메서드를 사용해서 필드별로 결과를 정렬할 수 있다.

예를 들어 위와 같이 모든 객체를 `title`로 정렬해 조회할 수 있다.

1행은 오름차순 설정이고, 3행의 `-` 음의 부호를 접두사로 사용해서 내림차순으로 나타낼 수 있다.

02. 블로그 어플리케이션 만들기

■ Django

- 모델용 관리 사이트 만들기
- QuerySet과 관리자로 작업하기
- QuerySet 객체 삭제하기 => SQL DELETE

```
>>> post = Post.objects.get(id=1)
>>> post.delete()
(1, {'blog.Post': 1})
```

객체를 삭제하기 위해서는 객체 인스턴스에서 `delete()` 메서드를 사용할 수 있다.

Thank You!



광운대학교
KwangWoon University